INSY7315



Imizamo Yethu

AFTERCARE MANAGEMENT SYSTEM

NovaTech

NovaTech Solutions

Table of Contents

Table of Figures ..... *Error! Bookmark not defined.*

Introduction *5*

Group Information *5*

User Roles *6*

User Stories *7*

Master Administrator (NovaTech Solutions) **7**

Administrator (Board Level – Sara) **7**

Manager (Current Aftercare Manager) **7**

Teacher (Teacher Per Class) **7**

Parent/Guardian **8**

UX Journey Map *9*

Work Breakdown Structure *13*

Non-Functional Requirements *14*

Performance **14**

Security **14**

Usability **14**

Reliability **15**

Scalability **15**

Maintainability **15**

Portability **15**

Analysis Artifacts *16*

Domain Class Diagram **16**

User Authentication and Role Specialization 16

Classroom Organization and Student Profiling 17

Daily Operations and Child Welfare Tracking 17

Permission and Stakeholder Reporting 18

Domain Summary 18

Design Class Diagram **20**

User Authentication and System Access Hierarchy 20

Classroom Organisation and Learner Entities 21

Verification and Reporting Classes 22

Design Class Summary 23

Implementation *25*

Sequence + State Diagrams **25**

Deployment and Client Handover Plan **9**

Introduction 9

Development environment 9

Staging environment 9

Production environment 9

Hosting overview 10

Deployment, Rollback, and Configuration 10

Client Handover and Responsibilities 11

Operational and Maintenance Guidelines 12

1. User Responsibilities 12

2. Database Management 12
Maintenance and Future Improvements 14
Client Training and Handover Acceptance 14
Conclusion 16

Data Schema *17*

Choice of Technology **17**
1. Purpose of the Document 17
2. Technology Requirements 17
3. Technology Evaluation Criteria 18
4. Application Architecture 18
5. Frontend Technology 19
5.2 Technologies Considered 19
6. Backend Technology 20
7. Database Technology 21
8. Hosting and Deployment 22
9. Authentication and Security 23
10. Reporting and Analytics Technology 24
11. Development and Collaboration Tools 24
12. Final Technology Stack 25
13. Conclusion 25

Entity Relationship Diagram **26**
User Authentication and Role Hierarchy 26
Classroom Structure and Learner Profiles 27
Operations and Safety Records 27
Verification and Reporting Modules 28
Schema Data Summary 28

Architecture *30*

Design Patterns **30**
1. Repository Pattern 30
2. Singleton Pattern 30
3. Middleware (Chain of Responsibility) 31

System Architecture Diagram **32**

Cloud Architecture Diagram **33**

Security *34*

Network Security **34**

Authentication and Authorisation **34**

Common Vulnerabilities and Mitigation Strategies **35**

Data Protection and Compliance Considerations **35**

Summary **36**

DevOps *37*

GitHub Actions Pipeline Diagram + Breakdown **37**
1. Branching Strategy 37
2. How the Pipeline Works 37
Why This Is Useful 37

Running Costs *39*

Selected Technology Stack **39**

Estimated Monthly Running Costs **39**
Azure Hosting Cost Calculation 39

Estimated Yearly Running Costs **40**

Projected Costs For 120 Users **40**

Future Growth Projection **41**
Conclusion **42**
Change Management Plan ***43***
Overview **43**
Objectives **43**
Stakeholder Adoption Strategy **43**
Adoption Strategy **44**
Phase 1 – Awareness 44
Phase 2 – Training 44
Phase 3 – Assisted Go-Live 45
Phase 4 – Full Adoption 45
Communication Strategy **45**
Feedback and Evaluation **46**
Risk Management During Adoption **46**
Handover Plan **47**
Training Plan **47**
Teacher Training 47
Manager Training 48
Parent Orientation **48**
Post-Handover Support **48**
Sustainability Plan **49**
Change Success Measures **49**
Declarations of Authenticity ***51***
POE Part 1 + 2 ..... **51**
Prototype ***54***
Figma Link **54**
Proof of Collaboration ***55***
References ***58***
AI Declaration ***58***

Figure 1: UX Journey Map - Teacher 9
Figure 2: UX Journey Map - Parent 10
Figure 3: UX Journey Map - Manager 11
Figure 4: UX Journey Map - Admin 12
Figure 5: UX Journey Map of Aftercare Management System 14
Figure 6: Work Breakdown Structure 13
Figure 7: Domain Class Diagram 16
Figure 8: Design Class Diagram 20
Figure 9: Sequence Diagram for Marking Attendance 25
Figure 10: Sequence Diagram for Recording a Developmental Observation 26
Figure 11: State Diagram of a Learner's Daily Life Cycle 27
Figure 12: Sequence Diagram for Donor Report Generation 28
Figure 13: Sequence Diagram for Learner Sign-Out 29
Figure 14: Table Showing The Hosting Overview 10
Figure 21: Entity Relationship Diagram 26
Figure 22: Repository Pattern 30
Figure 23: Singleton Pattern 30
Figure 24: UML Sequence Diagram of Middleware (Chain of Responsibility) 31
Figure 25: System Architecture Diagram 32
Figure 26: Cloud Architecture Diagram 33
Figure 28: GitHub Actions Pipeline Diagram 38
Figure 34: Team Shared Google Drive 55
Figure 35: Part One of Shared Progress Tracking Spreadsheet 56
Figure 36: Part 2 of Progress Tracking Spreadsheet 57

Introduction

The Imizamo Yethu Aftercare Management System is a community-focused software development project undertaken by a team of five Bachelor of Computer and Information Sciences in Application Development students as part of the Work Integrated Learning (WIL) module at Emeris. The project forms part of a software donation initiative aimed at supporting the operational needs of the Imizamo Yethu Aftercare, a non-profit organisation dedicated to providing educational support and a safe environment for children in the community.

Following discussions with key stakeholders, it was identified that the aftercare currently relies heavily on paper-based processes, particularly for attendance tracking, student sign-out procedures, and record keeping. These manual processes are time-consuming, difficult to manage, and present challenges in maintaining accurate records and ensuring child safety. Furthermore, teachers primarily have access to smartphones and possess varying levels of technical literacy, making simplicity, accessibility, and ease of use critical requirements for the proposed solution.

The purpose of the project is to design and develop a user-friendly web application that streamlines daily attendance management, improves learner safety through digital sign-out procedures, supports child development tracking, and provides reporting capabilities for management and donors. The system will be accessible across mobile and desktop devices, allowing teachers, management, and parents to engage more effectively with student information while reducing administrative workload. The solution will also assist the organisation in demonstrating its impact to donors through meaningful reports and analytics.

As a donated solution, this project seeks not only to fulfil the requirements of the Work Integrated Learning programme but also to create a sustainable and valuable digital platform that contributes to the long-term success and growth of the Imizamo Yethu Aftercare.

Group Information

Team Members:
ST10445500 – Isabelle Kathryn Devlin
ST10068995 – Thomas Pheiffer
ST10325652 – Michaela Ferraris
ST10301146 – Nomsa Mkoka
ST10280758 – Justin Fussell

User Roles

Role: Master Administrator
Who: NovaTech Solutions (dev team)
Primary Device: Desktop/Laptop
Key Responsibilities: System configuration, security settings, initial account setup for Admin/Manager, technical maintenance during handover.
Access Level: Full system access. After handover this account stays with the client's nominated representative, because only the Master Administrator can add Manager and Administrator accounts and reset forgotten staff passwords.

Role: Administrator
Who: Board-level (Sara)
Primary Device: Desktop/Laptop
Key Responsibilities: View organisation-wide reports and analytics for board/donor purposes, generates and exports PDF reports for donors.
Access Level: Reports and organisation-wide totals only, never individual learners; no daily attendance or data entry.

Role: Manager
Who: Aftercare Manager
Primary Device: Laptop/Phone
Key Responsibilities: Manages teacher and learner records across all 5 classes, oversees attendance data, generates and views PDF reports for statistics, manages classes and assigns their teachers, adds and suspends teacher accounts, and approves or dismisses flagged collections in the sign-out queue.
Access Level: Full operational access across all classes and learners.

Role: Teacher
Who: Class Teacher (5 classes, 20-30 learners each)
Primary Device: Phone
Key Responsibilities: Records daily attendance for any class (the register opens on their own class), logs learner development/wellbeing observations, performs digital sign out (verifying guardian against permissions).
Access Level: Teachers can view all classes and learners, and take attendance, record observations and sign learners out for any class, but only the Manager edits learner records. Simple UI required.

Role: Parent/Guardian
Who: Parent or Legal Guardian
Primary Device: Any device (Phone, Laptop, Desktop)
Key Responsibilities: View own child’s profile, development updates, and attendance history; views and adds authorised pickup people, and gives or withdraws permission to walk home alone.
Access Level: Restricted to their own child/children’s data only.

User Stories

Master Administrator (NovaTech Solutions)

ID	User Story
US-01	As a Master Administrator, I want to create and manage Administrator and Manager accounts, so that the organization can start using the system after handover.
US-02	As a Master Administrator, I want to configure system-wide security settings, so that learner data remains protected.
US-03	As a Master Administrator, I want full access to all modules during development and testing, so that I can verify the system works correctly before handover.

Administrator (Board Level – Sara)

ID	User Story
US-04	As an Administrator, I want to view organization-wide attendance statistics and reports, so that I can present accurate data to the board and donors.
US-05	As an Administrator, I want to generate and export PDF impact reports, so that I can share progress with current and potential donors.
US-06	As an Administrator, I want to manage high-level settings (classes, terms, school year), so that the system stays aligned with the aftercare structure.
US-07	As an Administrator, I want to oversee Manager accounts, so that operational access remains controlled.

Manager (Current Aftercare Manager)

ID	User Story
US-08	As a Manager, I want to view and manage all learner records across the 5 classes, so that I have complete oversight of the aftercare.
US-09	As a Manager, I want to view attendance data for all classes, so that I can identify trends and report accurately.
US-10	As a Manager, I want to generate PDF reports for donors and the board, so that I can support fundraising and accountability.
US-11	As a Manager, I want to manage teacher accounts, so that only authorized staff can access the system.
US-12	As a Manager, I want to handle escalations related to sign-outs, so that child safety issues can be resolved quickly.
US-13	As a Manager, I want a dashboard showing key statistics, so that I can make informed decisions quickly.

Teacher (Teacher Per Class)

ID	User Story
US-14	As a Teacher, I want to record daily attendance for my class using my phone, so that I can quickly mark learners present or absent.
US-15	As a Teacher, I want to search for a learner by name and toggle attendance, so that the process remains fast even when learners arrive at different times.

ID	User Story
US-16	As a Teacher, I want to record development and wellbeing observations for learners in my class, so that progress can be tracked over time.
US-17	As a Teacher, I want to rate learner progress in simple categories (e.g. mathematics, confidence, eye contact) on a 1–10 scale, so that observations remain consistent and easy to enter.
US-18	As a Teacher, I want to digitally sign out learners by verifying the guardian against registered permissions, so that child safety is maintained.
US-19	As a Teacher, I want to quickly see whether a learner is allowed to walk home alone, so that I do not make mistakes during busy collection times.
US-20	As a Teacher, I want to view and manage learners across all classes, so I can help wherever needed.
US-21	As a Teacher, I want the interface to have large, clear buttons and simple navigation, so that I can use the system easily on my smartphone.

Parent/Guardian

ID	User Story
US-22	As a Parent/Guardian, I want to view my child's profile and development updates, so that I can stay informed about their progress.
US-23	As a Parent/Guardian, I want to view my child's attendance history, so that I know how regularly they attend.
US-24	As a Parent/Guardian, I want to view registered pickup permissions, so that I can confirm who is allowed to collect my child.
US-25	As a Parent/Guardian, I want secure access limited only to my own child/children, so that other learners' information remains private.
US-26	As a parent/guardian, I want to register by confirming my child's name and date of birth, so that my account is automatically linked to the right learner.

Implementation status (November 2026): All user stories are implemented except where noted. US-02 (a security settings screen) and US-06 (terms and school years) were descoped; the Manager manages classes. US-07: only the Master Administrator manages Manager and Administrator accounts. US-17: observations use the categories Academic, Social, Wellbeing, Creative and Physical, each rated from 1 to 10. US-20: teachers can view every class and learner and take attendance, observations and sign-outs for any class, but only the Manager edits learner records. US-12 is met through the sign-out queue, where the Manager approves or dismisses collections that could not be verified.

UX Journey Map

The UX Journey maps below show how the different user roles interact with the Imizamo Yethu Aftercare system. The first map focuses on the user experience of Staff/Teachers, Parents, Managers and Admin users. They show the stages they move through, their actions, system touchpoints, thoughts and feelings, pain points, opportunities for improvement and emotional experience. The second user experience journey map shows a visual overview of the system screens and navigation. It shows how each user role moves between features such as dashboard(home screen), attendance, learner information, sign-out, classroom management and reporting. Both of these maps demonstrate both the users experience of the system and how the application's screens and functions to connect to support their different roles.

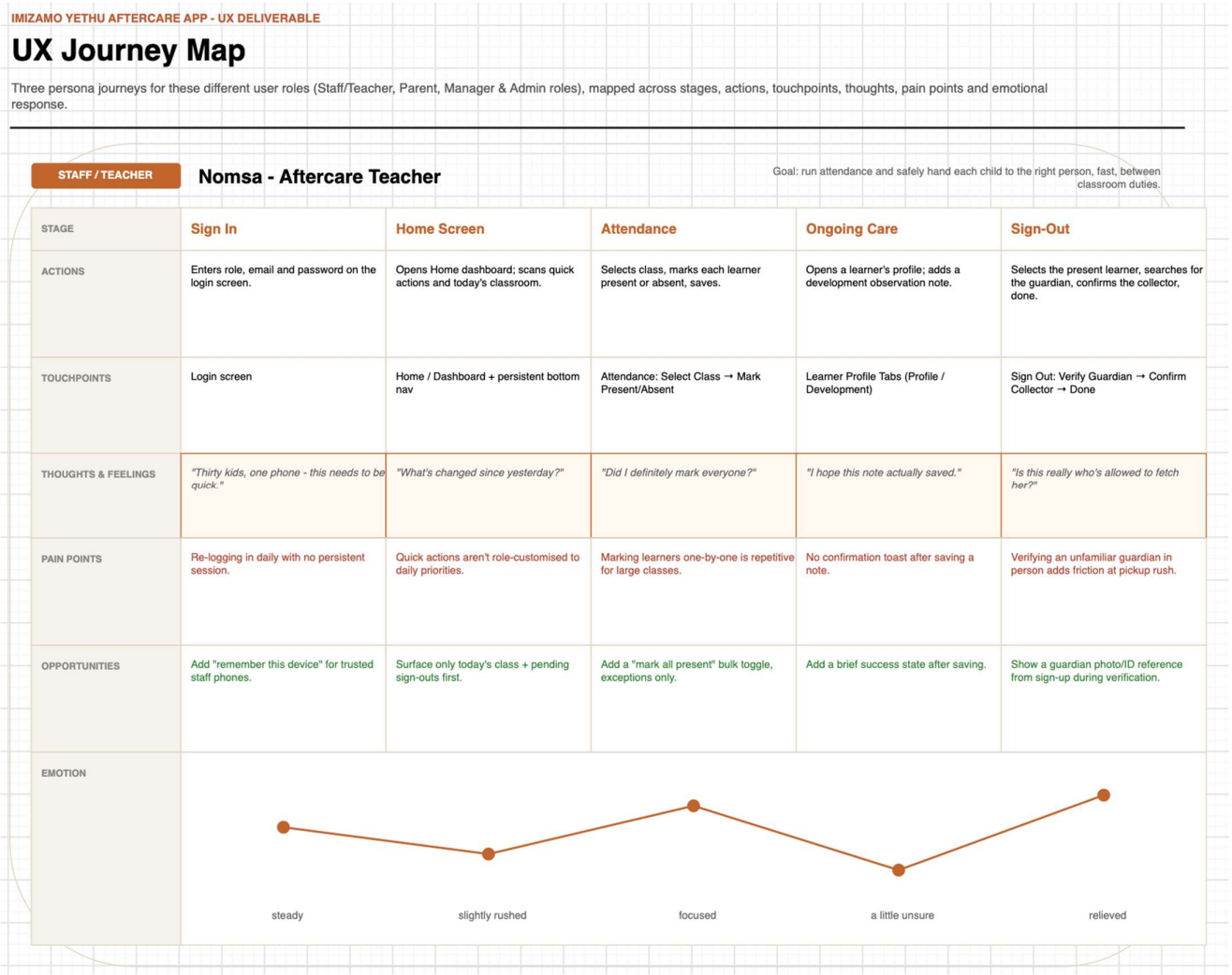


Figure 1: UX Journey Map - Teacher

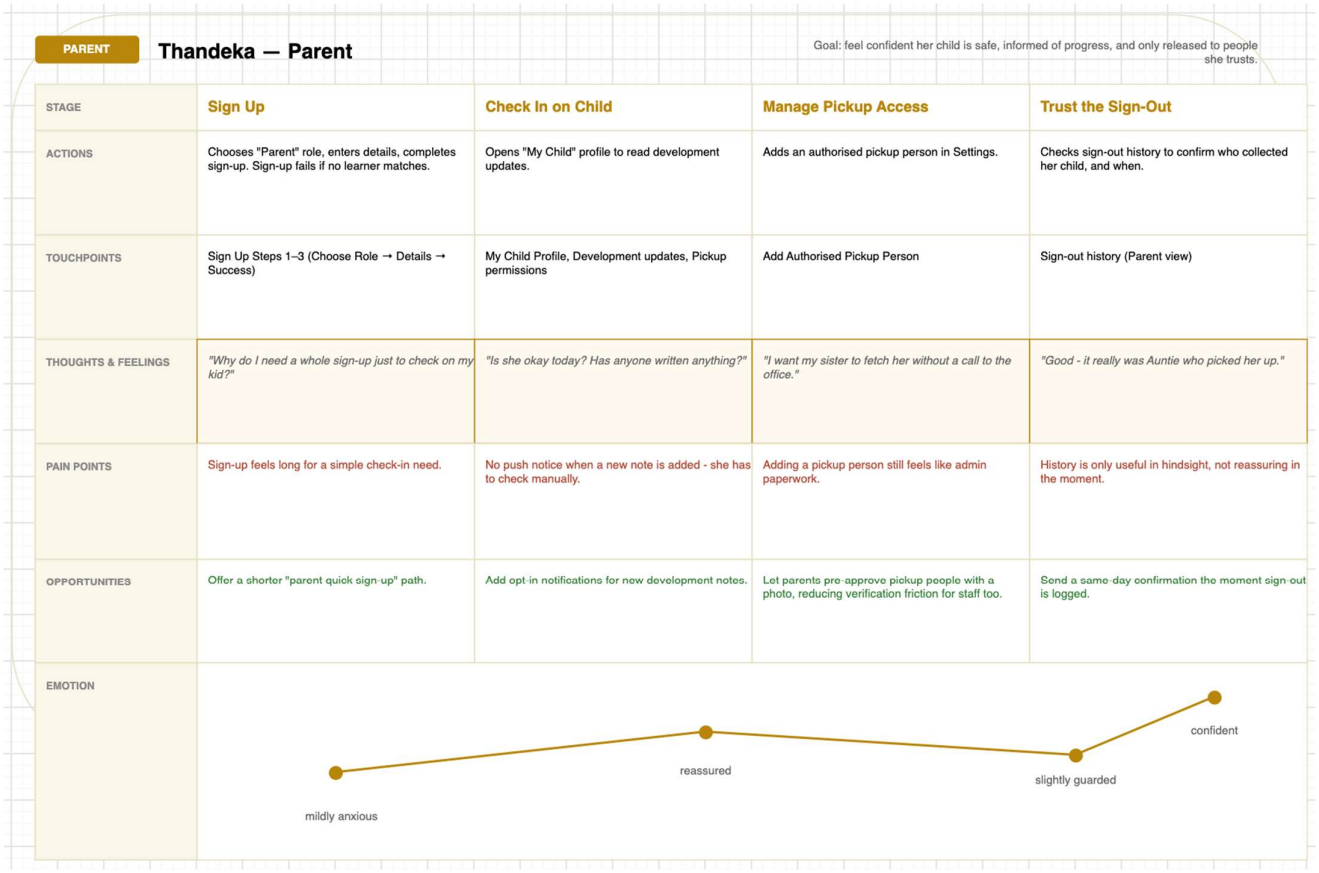


Figure 2: UX Journey Map - Parent

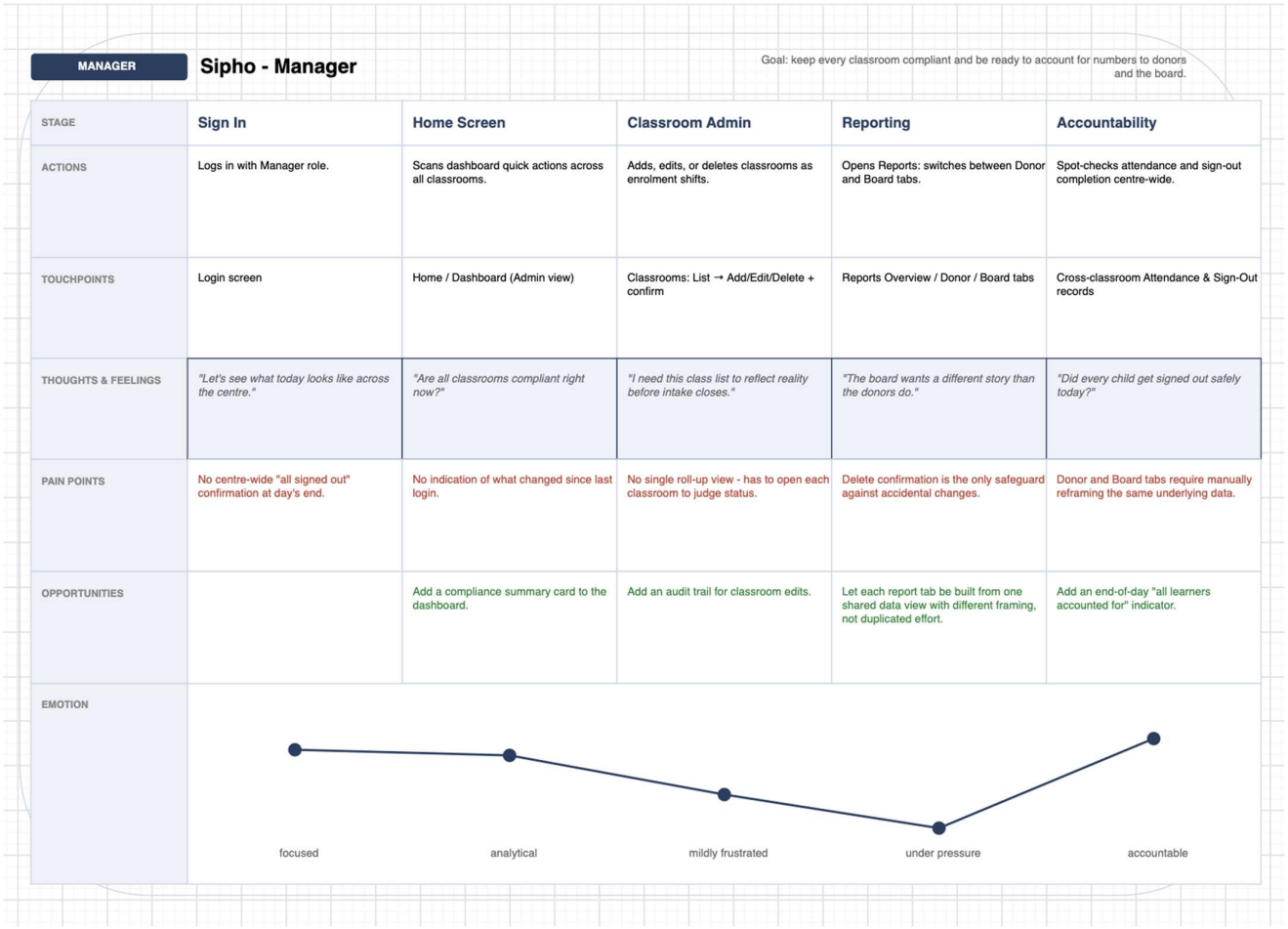


Figure 3: UX Journey Map - Manager

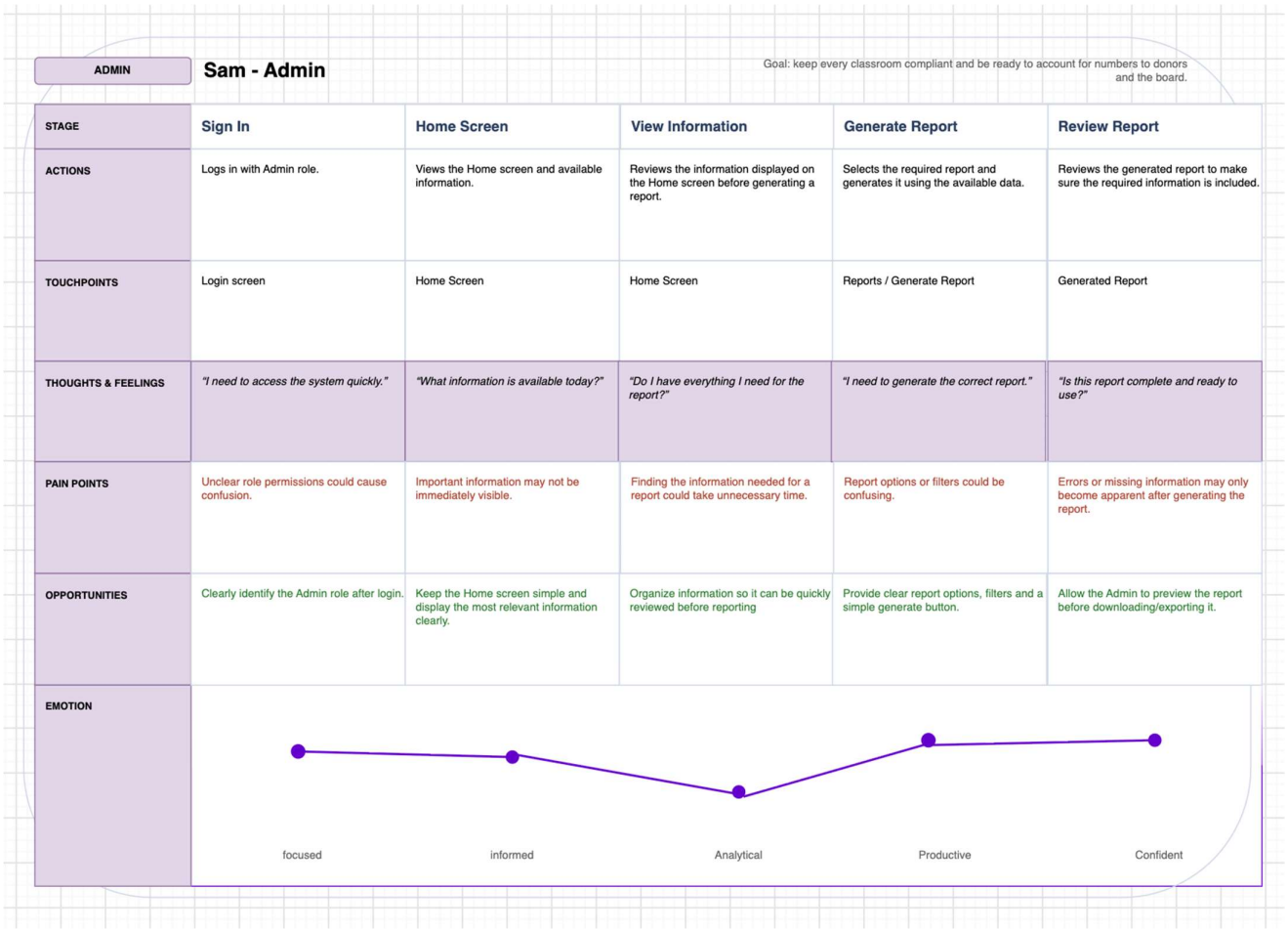


Figure 4: UX Journey Map - Admin

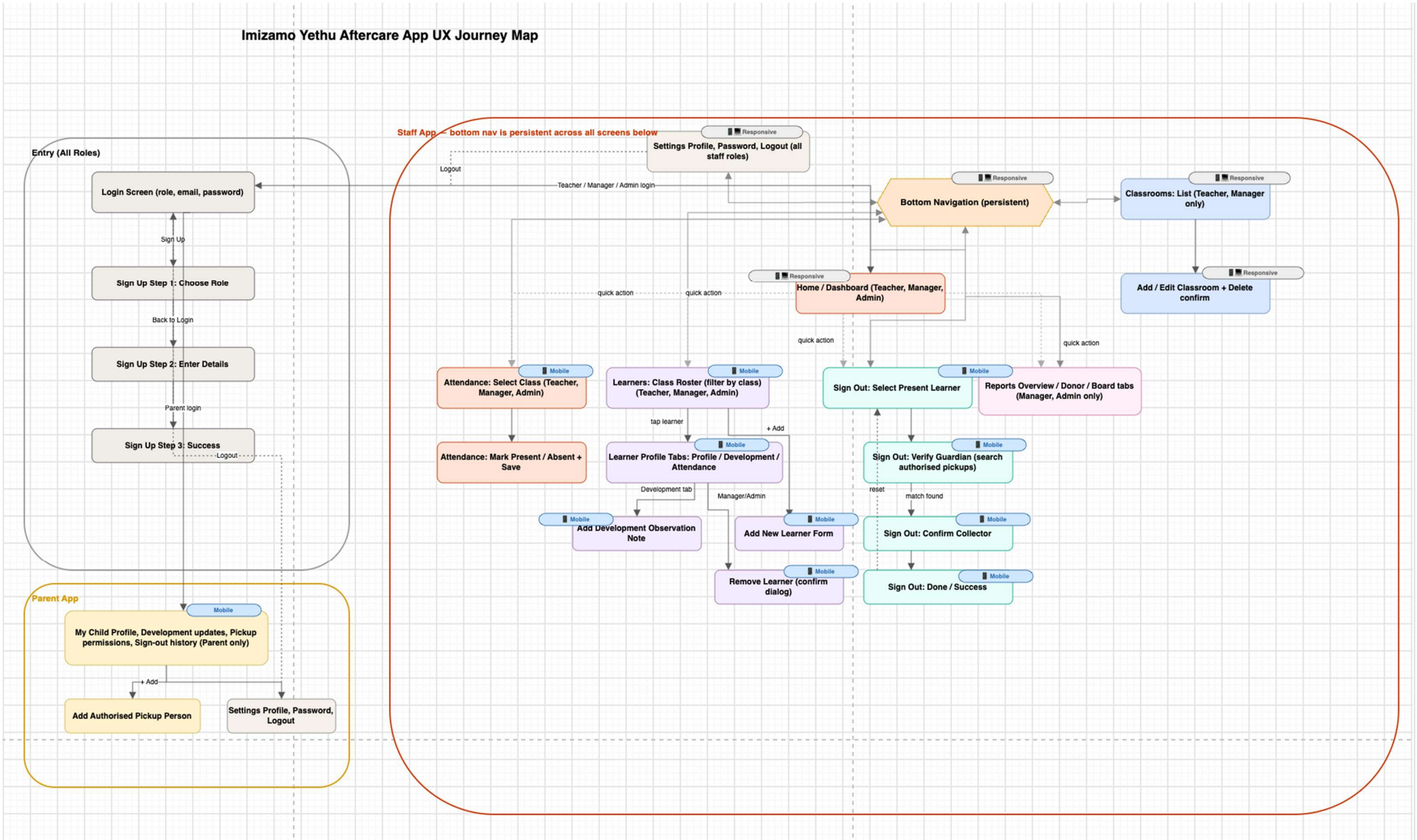


Figure 5: UX Journey Map of Aftercare Management System

Work Breakdown Structure

WBS ID	Task	Description	Duration	Predecessor	Responsible	Resources Needed
1.0	Cover Page	Team number, name, member names/numbers, client name & logo	5 days	None	Isabelle	Microsoft Word, client logo
4.0	User Roles	Identify and define system user roles	5 days	None	Justin, Thomas	Meeting minutes, Word
5.0	User Stories	Write user stories in standard format per role	5 days	4.0	Justin, Thomas	Microsoft Word
7.0	Work Breakdown Structure		5 days	None	Nomsa, Thomas	Excel / Word, Progress Tracker
8.0	Non-Functional Requirements	Document NFRs with justification	5 days	None	Michaela	Microsoft Word
13.0	Data Schema: Tech Choice	Decide & justify database/hosting technology	5 days	None	All	Group meeting
23.0	Prototype: Design	Visual layout, navigation, key screens	5 days	None	Michaela, Nomsa	Figma / IDE
6.0	UX Journey Map	Map screens & navigation per role	5 days	4.0, 5.0	All (Collaborative)	draw.io, Group meeting
9.0	Domain Class Diagram	UML domain model of core concepts	5 days	13.0?	Isabelle, Nomsa	draw.io
14.0	ERD	Entity relationship diagram + JSON schema if NoSQL	5 days	9.0, 13.0	All	draw.io / Word
15.0	Architecture: Design Patterns	Document design pattern choices	5 days	13.0	All	draw.io / Word
16.0	Architecture: Architecture Patterns	Document overall system structure pattern	5 days	13.0	All	draw.io / Word
17.0	Architecture: Cloud Architecture	Cloud services diagram & network segregation	5 days	13.0	All	draw.io / Word
19.0	DevOps: GitHub Pipeline Diagram	Branching strategy & CI/CD workflow diagram	5 days	13.0	Justin, Thomas	GitHub, draw.io
20.0	Running Costs	Monthly costs, 2-year projection, cost-at-scale table	5 days	13.0	Isabelle, Nomsa	Hosting provider pricing pages, Excel
21.0	Change Management	Adoption strategy, handover & training plan	5 days	None	Isabelle	Microsoft Word
24.0	Prototype: Flow of Information	Show data movement with dummy data	5 days	23.0	Michaela, Nomsa	Figma / IDE
10.0	Design Class Diagram	UML design class diagram (attributes, methods, keys)	5 days	9.0	Isabelle, Nomsa	draw.io
11.0	Sequence & State Diagrams	Critical flow diagrams	5 days	10.0	Justin	draw.io
12.0	Deployment Plan	Environment setup & rollout strategy	5 days	13.0	Michaela	Word, cloud provider docs
18.0	Security	XSS prevention, vulnerabilities, mitigation	5 days	14.0, 15.0, 16.0, 17.0	Michaela, Thomas	Microsoft Word
25.0	Prototype: Responsive Design	Mobile & desktop versions	5 days	24.0	Michaela, Nomsa	Figma
2.0	Table of Contents	Numbered sections, figures/tables	5 days	All content sections	Isabelle	Microsoft Word
3.0	Introduction	Justification of system for management	5 days	All content sections	Isabelle	Microsoft Word
22.0	Declaration of Authenticity	Signed by each group member	5 days	None	All	Module Manual Annexure A
26.0	Prototype Link Included	Figma/GitHub link in doc	5 days	23.0, 24.0, 25.0	All	ARC, Figma GitHub
27.0	Weekly Collaboration Sessions	Attendance & participation tracked weekly	5 days	None	All	Online/In-person meetings
28.0	Final Submission	One combined PDF submitted on ARC	1 day	1.0 - 26.0	Michaela/Isabelle	ARC

Figure 6: Work Breakdown Structure

Non-Functional Requirements

- **Performance:** speed and responsiveness
- **Security:** protection against unauthorized access
- **Usability:** ease of use
- **Reliability:** system stability and availability
- **Scalability:** ability to handle growth
- **Maintainability:** ease of updates and fixes
- **Portability:** ability to run in different environments

Performance

Requirement: The system should respond to user actions in a few seconds under normal operating conditions.
Justification: Quick response times will allow the teachers to complete tasks efficiently without delays, especially during busy aftercare hours when information may need to be accessed quickly. For example, when children are being collected, teachers need to quickly identify who is collecting each child and whether that person has permission to do so, or whether the child has permission to leave independently.

Requirement: The system should be mobile responsive and function effectively on different screen sizes.
Justification: Teachers / other users may need to access the system from different devices, such as laptops/desktops, tablets or mobiles phones. Mobile responsiveness will allow the system to remain accessible and usable regardless of the device being used.

Requirement: The system should operate using the aftercares existing WIFI network
Justification: Using the existing network will reduce the need for additional infrastructure and costs while allowing teachers to access the system using the resources already available to the aftercare.

Security

Requirement: The system should securely store and protect sensitive learner information.
Justification: Learner records may contain personal and confidential information. Appropriate security measures are therefore necessary to prevent unauthorized access, modification or disclosure of the data.

Requirement: The system should support multiple user roles.
Justification: Different users may have different responsibilities and should only have access to the functions and information relevant to their role. This will improve both security and usability.

Usability

Requirement: The system must be easy to learn and use
Justification: A simple and intuitive system will reduce the amount of training required and allow teachers to adopt the system more easily as part of their existing daily activities.

Requirement: The system should require minimal technical knowledge from teachers and other users.
Justification: Users may have different levels of technical experience. The system should therefore allow teachers to complete their required tasks without needing advanced computer skills.

Reliability

Requirement: The system should be reliable and available during aftercare hours.
Justification: Teachers may depend on the system to access and record important information throughout the day. Reliable availability will reduce disruptions to aftercare operations.

Scalability

Requirement: The system should be capable of supporting a large number of users with minimal downtime or reduction in performance.
Justification: The number of users using the system may increase over time. Designing the system to support additional users will allow it to continue operating effectively as the aftercare grows.

Maintainability

Requirement: The system should be designed so that updates, bug fixes and changes can be made without significantly disrupting its operation.
Justification: The needs of the aftercare may change over time. A maintainable system will make it easier to correct problems and introduce improvements without having to rebuild the entire system.

Portability

Requirement: The system should be accessible through commonly used web browsers on computers, tablets and mobile devices.
Justification: Teachers may use different devices to access the system. Browser-based accessibility will reduce dependence on specific hardware or operating systems and make the system more accessible to the aftercare.

Analysis Artifacts

Domain Class Diagram

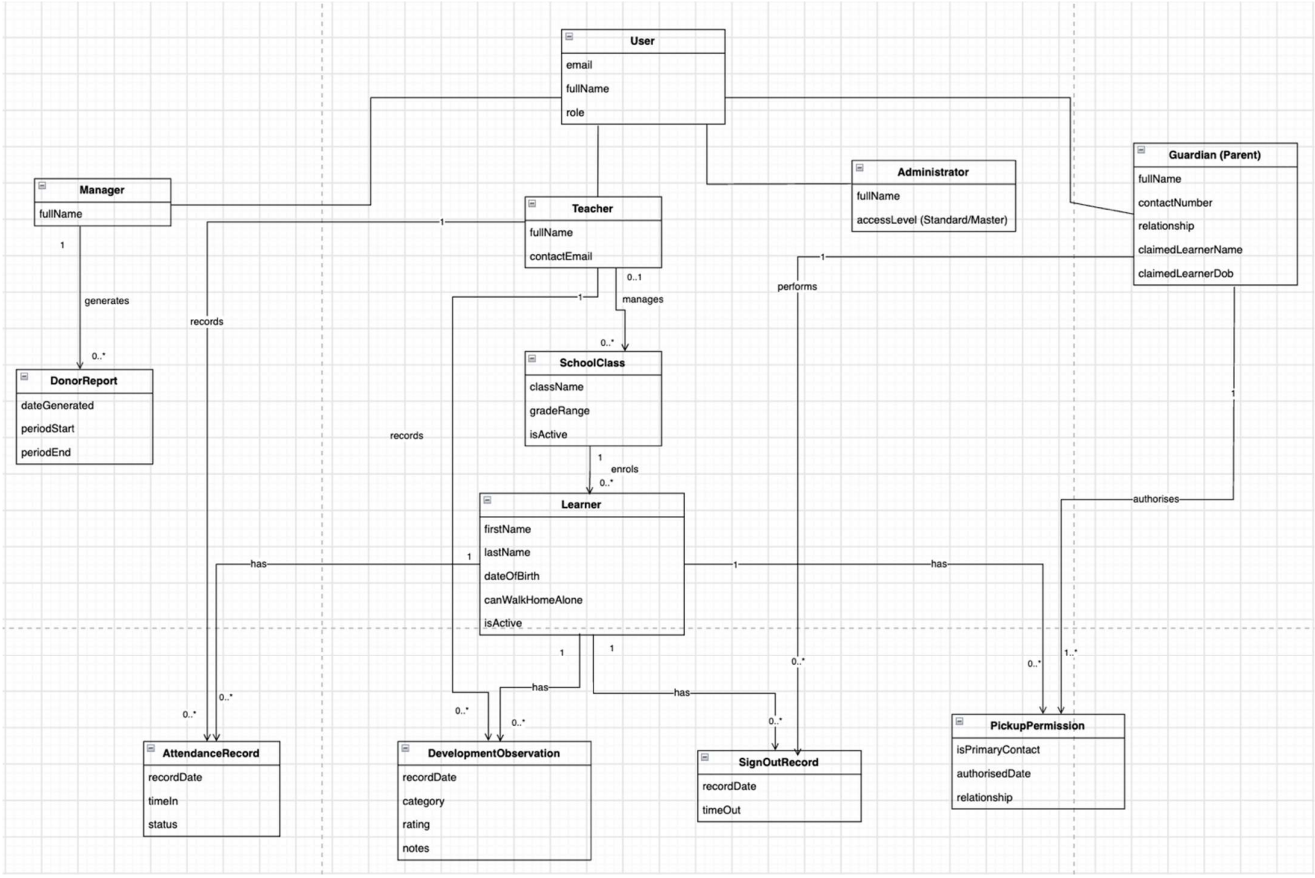


Figure 7: Domain Class Diagram

This UML Class diagram presents the domain model for the Imizamo Yethu Aftercare Management System. Unlike physical database schemas this diagram focuses specifically on high-level domain entities, their descriptive attributes and operational relationships without depicting technical primary or foreign key variables.

User Authentication and Role Specialization

This model utilizes inheritance also known as generalization from a central parent entity to handle identity management across user roles:

User (The Base Domain Entity)

- Attributes: email, fullName, role
- Serves to capture core identity information shared across all platform users

Administrator

- Attributes: fullName, accessLevel (Standard/Master)
- Serves to handle platform governance and high-level organizational oversight

Manager

- Attributes: fullName
- Serves to oversee daily aftercare operations across all classrooms manages donor reporting
- Relationships: Generates 0..* DonorReport entities

Teacher

- Attributes: fullName, contactEmail

- This facilitates mobile-first classroom data logging and daily child management
- Relationships:
  - Manages 0..* SchoolClass entities
  - Manages 0..* AttendanceRecord entries
  - Records 0..* DevelopmentObservation entries

Guardian (Parent)

- Attributes: fullName, contactNumber, relationship, claimedlearnerName, claimedLearnerDob
- This captures parent/contact profiles for child collection authorisations, observations and progress tracks
- Relationships:
  - Performs 0..* SignOutRecord verification actions

Classroom Organization and Student Profiling

SchoolClass

- Attributes: className, gradeRange, isActive
- Represents a classroom group structured for mobile navigation
- Relationships: Managed 0..1 Teacher; enrolls 0..* Learner entities

Learner

- Attributes: firstName, lastName, dateofBirth, canWalkHome, isActive
- This is the central profile for a child enrolled in the aftercare program
- Relationships:
  - Enrolled by 1 SchoolClass
  - Has 0..* AttendanceRecord logs
  - Has 0..* DevelopmentObservation updates
  - Has 0..* SignOutRecord departure logs
  - Has 0..* PickupPermission collection rules

Daily Operations and Child Welfare Tracking

AttendanceRecord

- Attributes: recordDate, timeIn, status
- This replaces the manual registering in books process with a digital daily check-in logs
- Relationships: Recorded by 1 Teacher and 1 Learner

DevelopmentObservation

- Attributes: recordDate, category, rating, notes
- This captures the qualitative assessment regarding learner behaviour, wellbeing and milestone progress
- Relationship: Recorded by 1 Teacher for 1 Learner

SignOutRecord

- Attributes: recordDate, timeOut
- This provides a digital audit log for a safe child collection
- Relationships: Captures the multi-way interaction between 1 Learner, 1 Guardian (Parent) and 1 Teacher who is performing the sign-out action

---

Permission and Stakeholder Reporting

PickupPermission

- Attributes: isPrimaryContact, authorisedDate,relationship
- This validates collection eligibility before releasing a child
- Relationships: Authorised by 1 Guardian (Parent) and linked to 1 Learner

DonorReport

- Attributes: dateGenerated, periodStart, periodEnd
- This captures metadata for exporting performance metrics and PDF analytics
- Relationships: Generated by 1 Manager

Domain Summary

Entity	Domain Attributes	Primary Responsibility
User	email, fullName, role	Base user identity
Administrator	fullName, accessLevel	High-level system settings
Manager	fullName	Operations and donor report creation
Teacher	fullName, contactEmail	Daily attendance and observation logging
Guardian (Parent)	fullName, contactNumber, relationship, claimedLearnerName, claimedLearnerDob	Child access and pickup permissions
SchoolClass	className, gradeRange, isActive	Class grouping
Learner	firstName, lastName, dateOfBirth, canWalkHomeAlone, isActive	Core child profile
AttendanceRecord	recordDate, timeIn, status	Daily digital check-in

DevelopmentObservation	recordDate, category, rating, notes	Qualitative child progress notes
SignOutRecord	recordDate, timeOut	Departure security log
PickupPermission	isPrimaryContact, authorisedDate, relationship	Verified collector list
DonorReport	dateGenerated, periodStart, periodEnd	Aggregated report metadata

---

Design Class Diagram

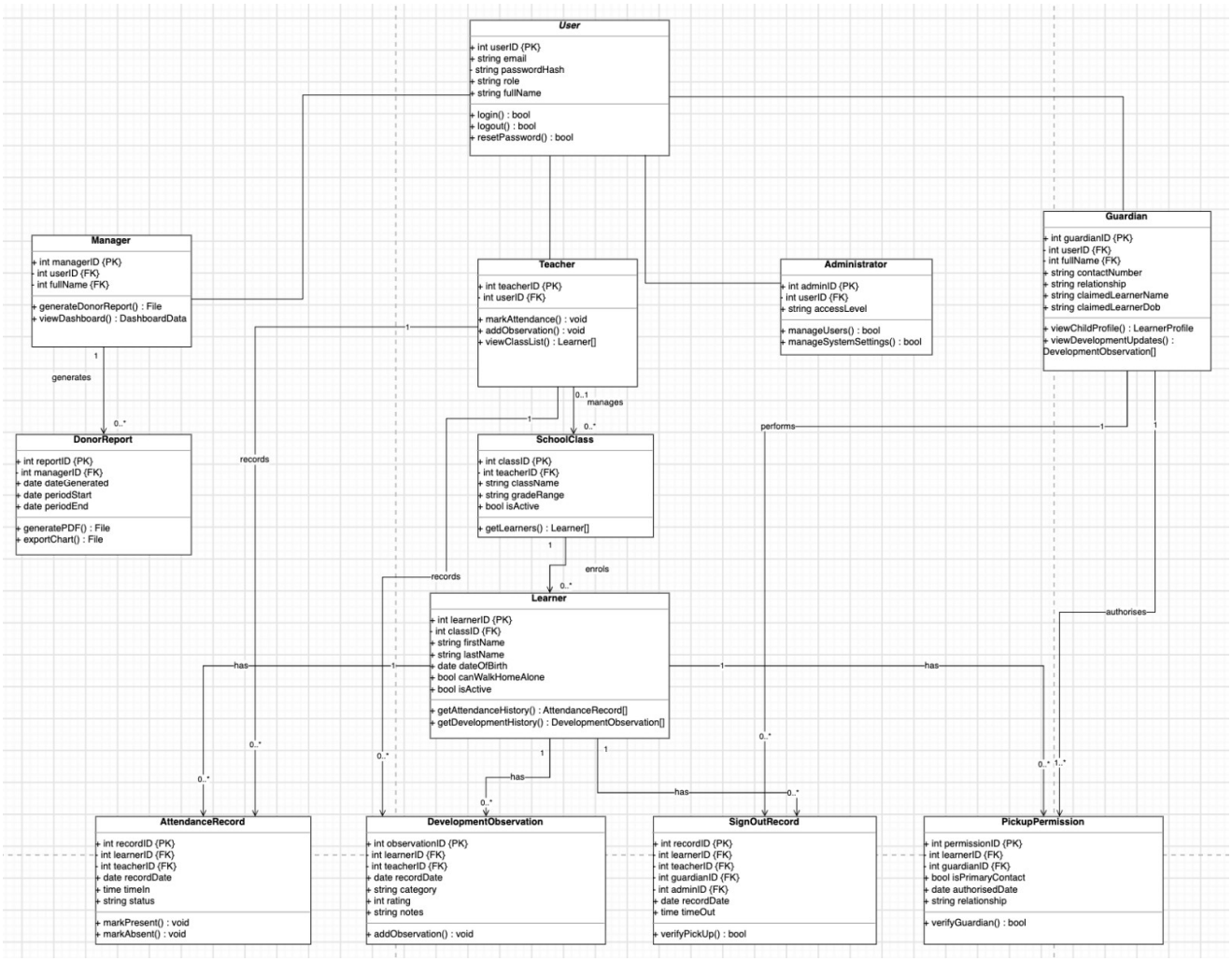


Figure 8: Design Class Diagram

This UML Class Diagram represents the implementation-level (Detailed Design) architecture for the Imizamo Yethu Management System. Unlike the conceptual or logical schemas this UML diagram explicitly models object-orientated practices including field visibility such as public or private, int, string, date, bool and object arrays as well as foreign key structures and system operations/methods.

User Authentication and System Access Hierarchy

The design employs standard class inheritance under a centralized abstract/base User entity:

User (Base Entity Class)

- Attributes: + int userID {PK}, + string email, - string passwordHash, +string role, + string fullName
- Methods:
  - + login() : bool
  - + logout() : bool
  - + resetPassword() : bool
- This serves to encapsulate authentication routines and base identity traits shared across all system actors

Administrator

- Attributes: + int adminID{PK}, - int userID {FK}, + string accessLevel

- Methods:
    - + manageUsers() : bool
    - + manageSystemSettings() : bool
- This governs any high-level system preferences, master accounts and security access

Manager

- Attributes: +int managerID {PK}, - int userID {FK}
- Methods:
    - + generateDonorReport() : File
    - + viewDashboard() : DashboardData
- This oversees program operations and compiles administrative donor analytics
- Relationships: Generates 0..* DonorReport instances

Teacher

- Attributes: + int teacherID {PK}, - int userID{FK}, + string contactEmail
- Methods:
    - + markAttendance() : void
    - + addObservation() : void
    - - viewClassList() : Learner[]

- This provides classroom management methods designed for quick touch interfaces on mobile devices
- Relationships: Manages 0..* SchoolClass instances; record 0..* AttendanceRecord and DevelopmentObservation entries

Guardian

- Attributes: +int guardianID{PK}, - int userID{FK}, + string contactNumber, + string relationship, + string claimedLearnerName, + string claimedLearnerDob
- Methods:
    - + viewChildProfile() : LearnerProfile
    - + viewDevelopmentUpdates() : DevelopmentObservation[]

- This enables parent access to personal child profile metrics and collection authorisations
- Relationships: Authorise 1..* PickupPermission entries; participates in 0..* SignOutRecord activities

---

Classroom Organisation and Learner Entities

SchoolClass

- Attributes: + int classID {PK}, - int teacherID {FK}, + string className, +string gradeRange
- Methods:
    - + getLearners() : Learner[]
- Purpose: Groups children into manageable class units
- Relationships: Managed by 0..1 Teacher: enrolls 0..* Learner objects

Learner

- Attributes: + int learnerID{PK}, - int classID{FK}, + string firstName, + string lastName, + date dateOfBirth, + bool canWalkHomeAlone
- Methods:
    - + getAttendanceHistory() : AttendanceRecord[]
    - + getDevelopmentHistory() : DevelopmentObservation[]

- This serves as the central domain model holding profile data and historical data accessors for each enrolled child
- Relationships:
    - Enrolled 1 SchoolClass
    - Has 0..* AttendanceRecord entries
    - Has 0..* DevelopmentObservation entries
    - Has 0..* SignOutRecord entries
    - Has 0..* PickupPermission rules

AttendanceRecord

- Attributes: + int recordID{PK}, - int learnerID {FK}, - int teacherID {FK}, + date dateRecordDate, + time timeIn, + string status
- Method:
    - +markPresent() : void
    - +markAbsent() : void
- This encapsulates the daily classroom check-in logic

DevelopmentObservation

- Attributes: + int observationID{PK}, - int learnerID {FK}, - int teacherID {FK}, + date dateRecordDate, + string category, + int rating, + string notes
- Methods:
    - +addObservation() : void
- This captures the qualitative assessments and development scores for learners

SignOutRecord

- Attributes: + int recordID{PK}, - int learner{FK}, - int teacherID {FK}, - int guardian {FK}, + date dateRecordDate, + time timeOut
- Methods:
- +verifyPickUp() : bool

Verification and Reporting Classes

PickupPermission

- Attributes: + int permissionID{PK}, - int learnerID {FK}, - int guardianID {FK}, + bool isPrimaryContact, + date dateAuthorisedDate
- Methods:
    - +verifyGuardian() : bool
- This evaluates whether a guardian is authorised to pick up a given child

DonorReport

- Attributes: + int reportID {PK}, - int managerID {FK}, date dateGenerated, + date periodStart, + date periodEnd
- This manages the performance analytics by generating exportable PDF reports for external board members and donors
- Methods:
    - +generatePDF() : File
    - +exportChart() : File

Design Class Summary

Class Name	Attributes & Visibility	System Methods / Operations	Responsibilities
User	userID (PK), email, passwordHash, role, fullName	login(), logout(), resetPassword()	Core authentication base class
Administrator	adminID (PK), userID (FK), accessLevel	manageUsers(), manageSystemSettings()	Platform & security configuration
Manager	managerID (PK), userID (FK)	generateDonorReport(), viewDashboard()	Operations overview & donor reporting
Teacher	teacherID (PK), userID (FK), contactEmail	markAttendance(), addObservation(), viewClassList()	Mobile-first daily classroom logger
Guardian	guardianID (PK), userID (FK), contactNumber, relationship, claimedLearnerName, claimedLearnerDob	viewChildProfile(), viewDevelopmentUpdates()	Parent portal child access
SchoolClass	classID (PK), teacherID (FK), className, gradeRange, isActive	getLearners()	Classroom cohort grouping

Learner	learnerID (PK), classID (FK), firstName, lastName, dateOfBirth, canWalkHomeAlone, isActive	getAttendanceHistory(), getDevelopmentHistory()	Child profile & data aggregator
AttendanceRecord	recordID (PK), learnerID (FK), teacherID (FK), dateRecordDate, timeIn, status	markPresent(), markAbsent()	Digital daily attendance logging
DevelopmentObservation	observationID (PK), learnerID (FK), teacherID (FK), dateRecordDate, category, rating, notes	addObservation()	Qualitative wellbeing tracking
SignOutRecord	recordID (PK), learnerID (FK), teacherID (FK), guardianID (FK), dateRecordDate, timeOut	verifyPickUp()	Safe child checkout verification
PickupPermission	permissionID (PK), learnerID (FK), guardianID (FK), isPrimaryContact, dateAuthorisedDate	verifyGuardian()	Guardian collection authorisation
DonorReport	reportID (PK), managerID (FK), dateGenerated, periodStart, periodEnd	generatePDF(), exportChart()	Analytics PDF & chart generation

Implementation

Sequence + State Diagrams

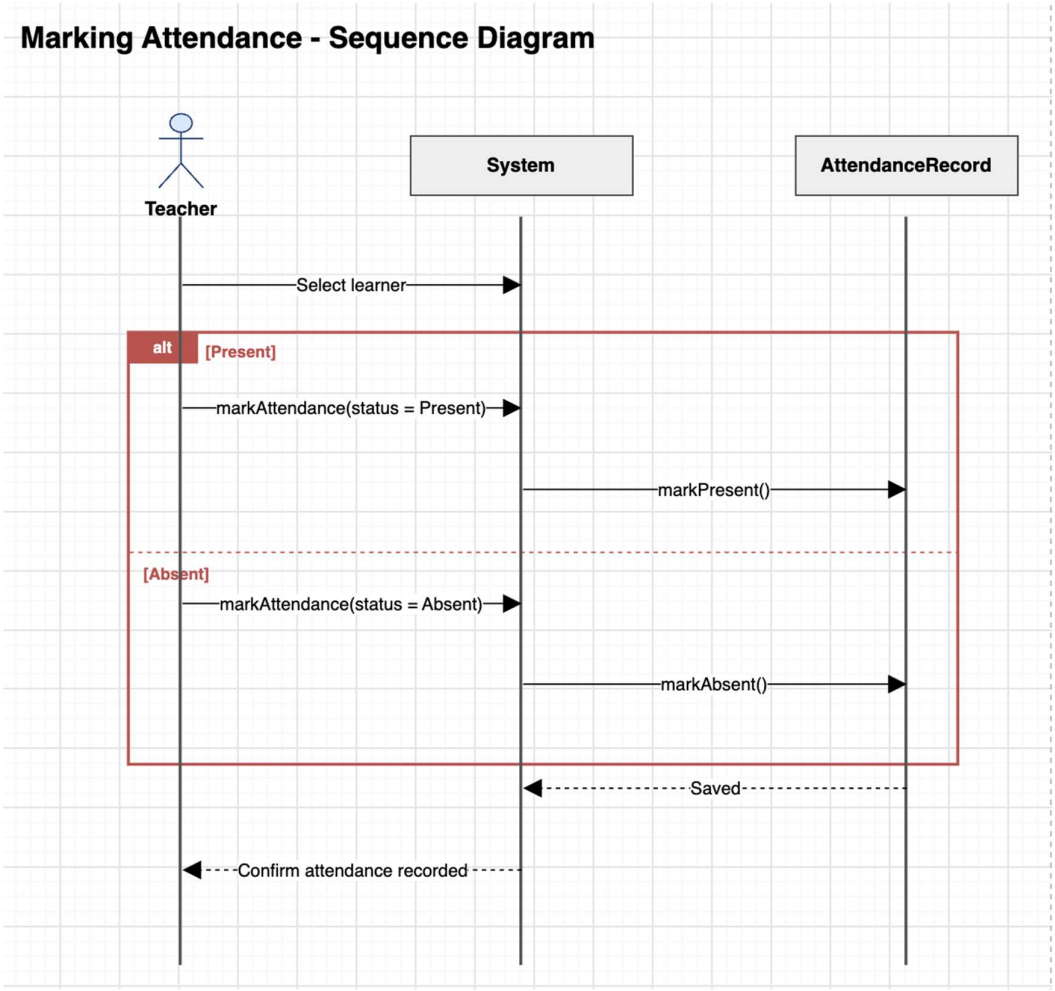


Figure 9: Sequence Diagram for Marking Attendance

This UML sequence diagram shows the process of marking attendance for a learner by a teacher.
Diagram elements:

- Actors & Objects:
  - Teacher: Human user initiating the attendance process.
  - System: The main software interface that handles user input and orchestration.
  - AttendanceRecord: The database component or backend entity managing attendance data storage.

- Initial Action:
  - The teacher selects a learner on the screen by sending the message Select learner to the system.

- Conditional Block (alt Frame):
  - Scenario 1: [Present]
    1. The Teacher sends the message markAttendance(status = Present)
    2. The System forwards this instruction by invoking the method markPresent() on the AttendanceRecord.
  - Scenario 2 : [Absent]

1. The Teacher sends the message markAttendance(status = Absent) to the System.
2. The System forwards this instruction by invoking the method markAbsent() on the AttendanceRecord.

- Confirmation Sequence:
  1. The AttendanceRecord responds to the System with a dashed message: Saved.
  2. The System communicates back to the Teacher with a final confirmation: Confirm attendance recorded.

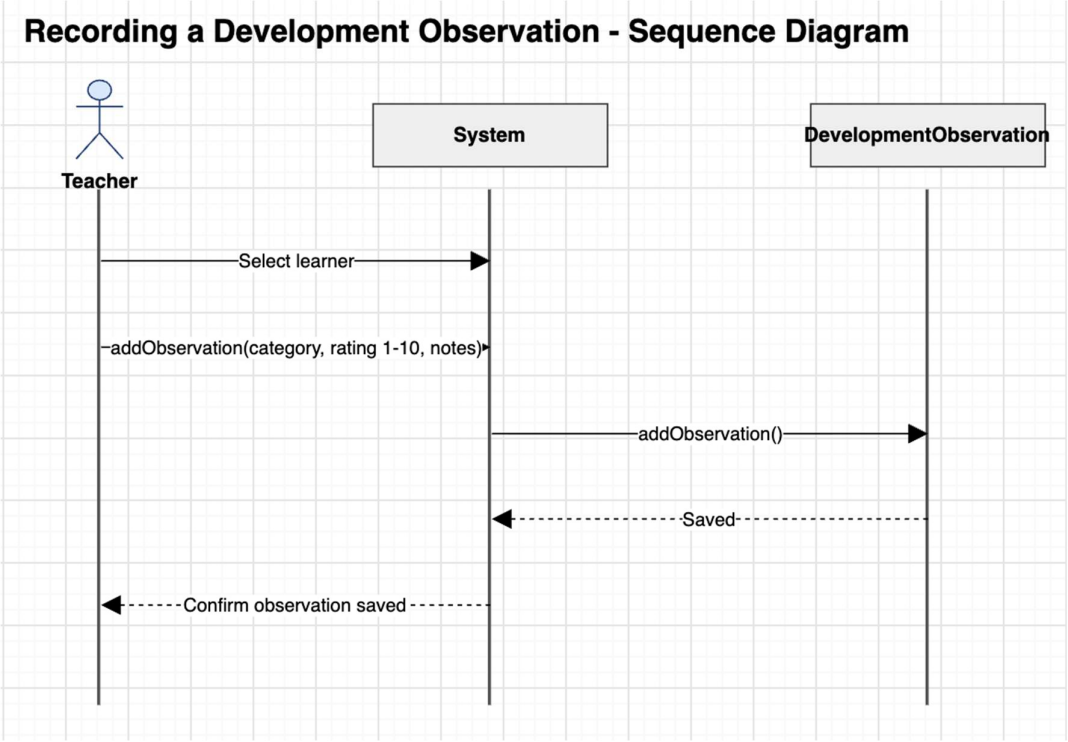


Figure 10: Sequence Diagram for Recording a Developmental Observation

This UML sequence diagram shows the interaction flow for Recording a Development Observation for a learner.

Diagram Elements

- Actors and Objects:
  - Teacher: The main actor initiating and submitting the observation.
  - System: The user interface and underlying application logic.
  - DevelopmentObservation: The backend entity/database is responsible for storing development record data.

Interaction Steps

1. Learner Selection: The Teacher starts the process by sending the Select learner message to the System.
2. Observation Details Submission: The Teacher inputs data and sends the addObservation(category, rating 1-10, notes) message to the System.
3. Data Persistence: The System processes the request and calls addObservation() on the DevelopmentObservation entity.

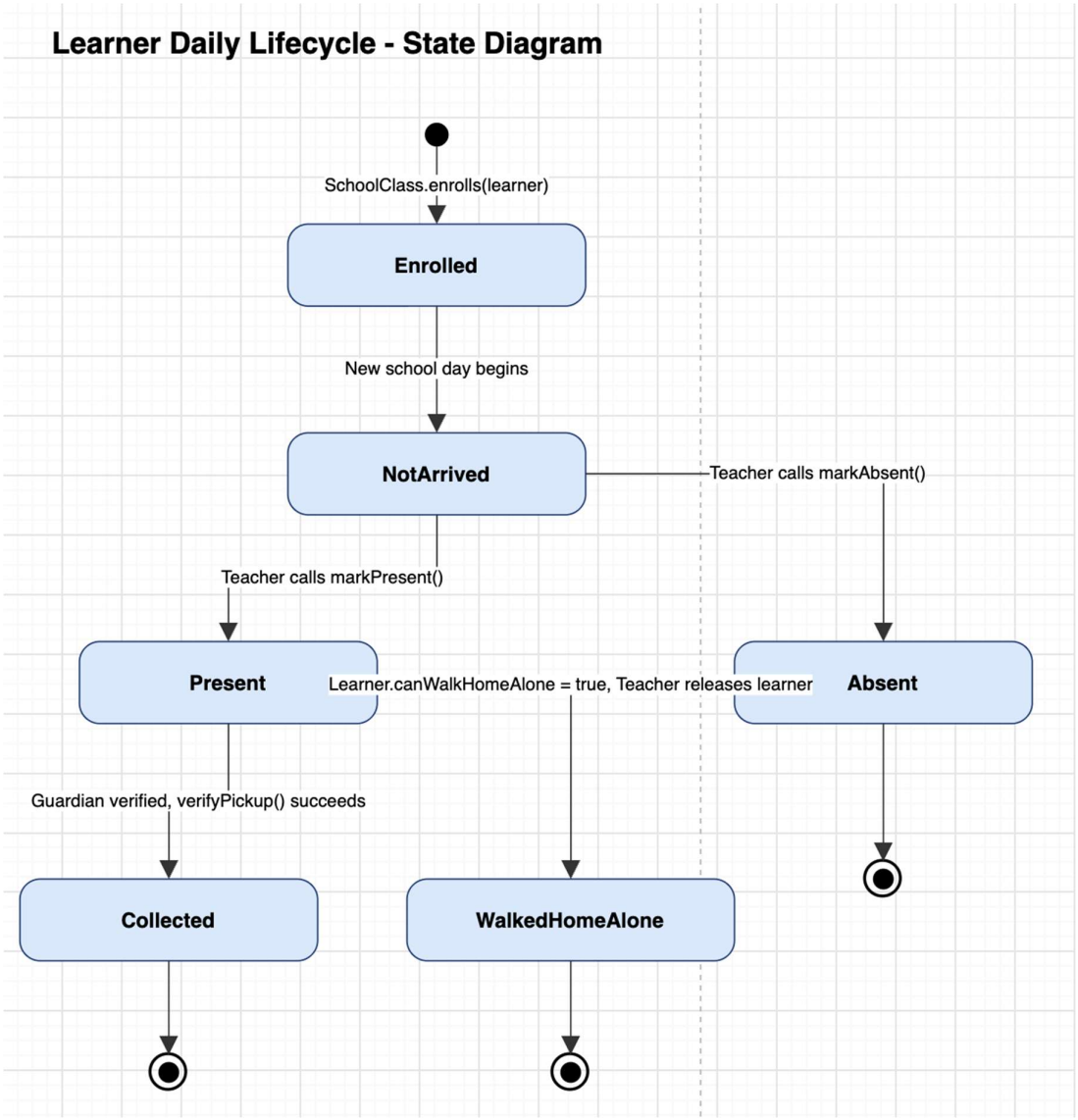


Figure 11: State Diagram of a Learner's Daily Life Cycle

This UML state diagram shows the Learner Daily Lifecycle. It traces a student's states from initial enrollment through their daily attendance and signout.

States and Transitions:

- Initial State to Enrolled: The process starts when SchoolClass.enrolls(learner) is triggered, then it places the learner into the Enrolled state.
- Enrolled to NotArrived: When a New school day begins, the learner enters the default NotArrived state for the day.

Attendance Branching (NotArrived)

- Absent Path: If the Teacher calls markAbsent(), the learner moves to the absent state. This learner will then immediately reach the final state (shown as the bullseye icon).
- Present Path: If the Teacher calls markPresent(), the learner transitions to the Present state.

Dismissal Branching (Present)

- Collected: Transition occurs when Guardian verified, verifyPickup() succeeds, moving the learner to Collected, which then leads to the final state.
- WalkedHomeAlone: Transition happens when Learner.canWalkHomeAlone = true, Teacher releases learner, moving the learner to WalkedHomeAlone, which leads to the final state.

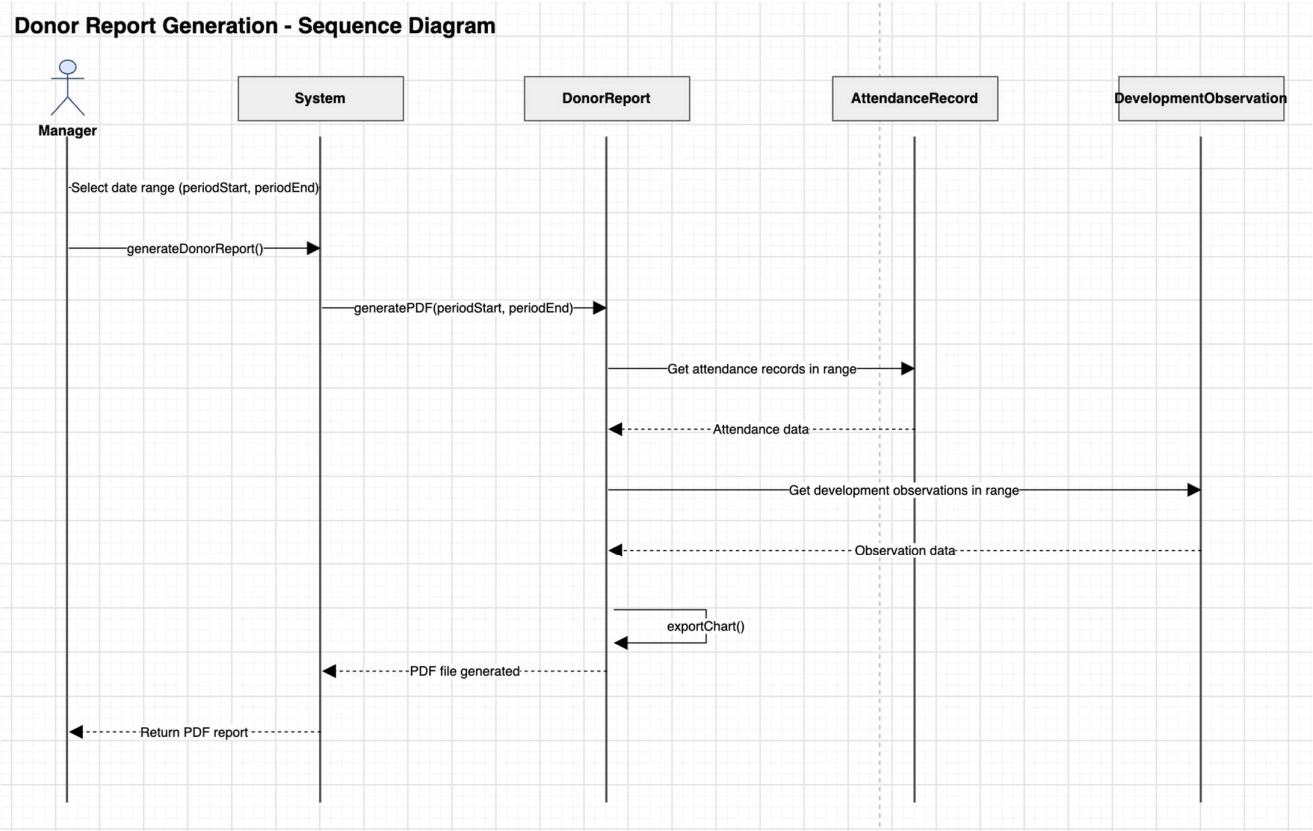


Figure 12: Sequence Diagram for Donor Report Generation

This sequence diagram, titled Donor Report Generation - Sequence Diagram, details the technical flow for compiling and exporting donor reports.

Participating Entities:

- Actor: Manager
- System Components: System, DonorReport, AttendanceRecord, DevelopmentObservation

Process Flow

1. Date Range Selection: The Manager inputs the parameter 'Select date range (periodStart, periodEnd)' into the System.
2. Report Initiation: The Manager calls 'generateDonorReport()' on the System.
3. PDF Request: The System forwards a 'generatePDF (periodStart, periodEnd)' call to the DonorReport module.
4. Attendance Data Fetch: DonorReport queries AttendanceRecord with 'Get attendance records in range' and receives the returned 'Attendance data'.
5. Observation Data Fetch: DonorReport queries DevelopmentObservation with 'Get development observations in range' and receives the returned 'Observation data'.
6. Chart Rendering: DonorReport triggers an internal self-call 'exportChart()' to generate required visualisations.
7. File Generation & Delivery: DonorReport notifies the System with 'PDF file generated', which then executes 'Return PDF report' back to the Manager.

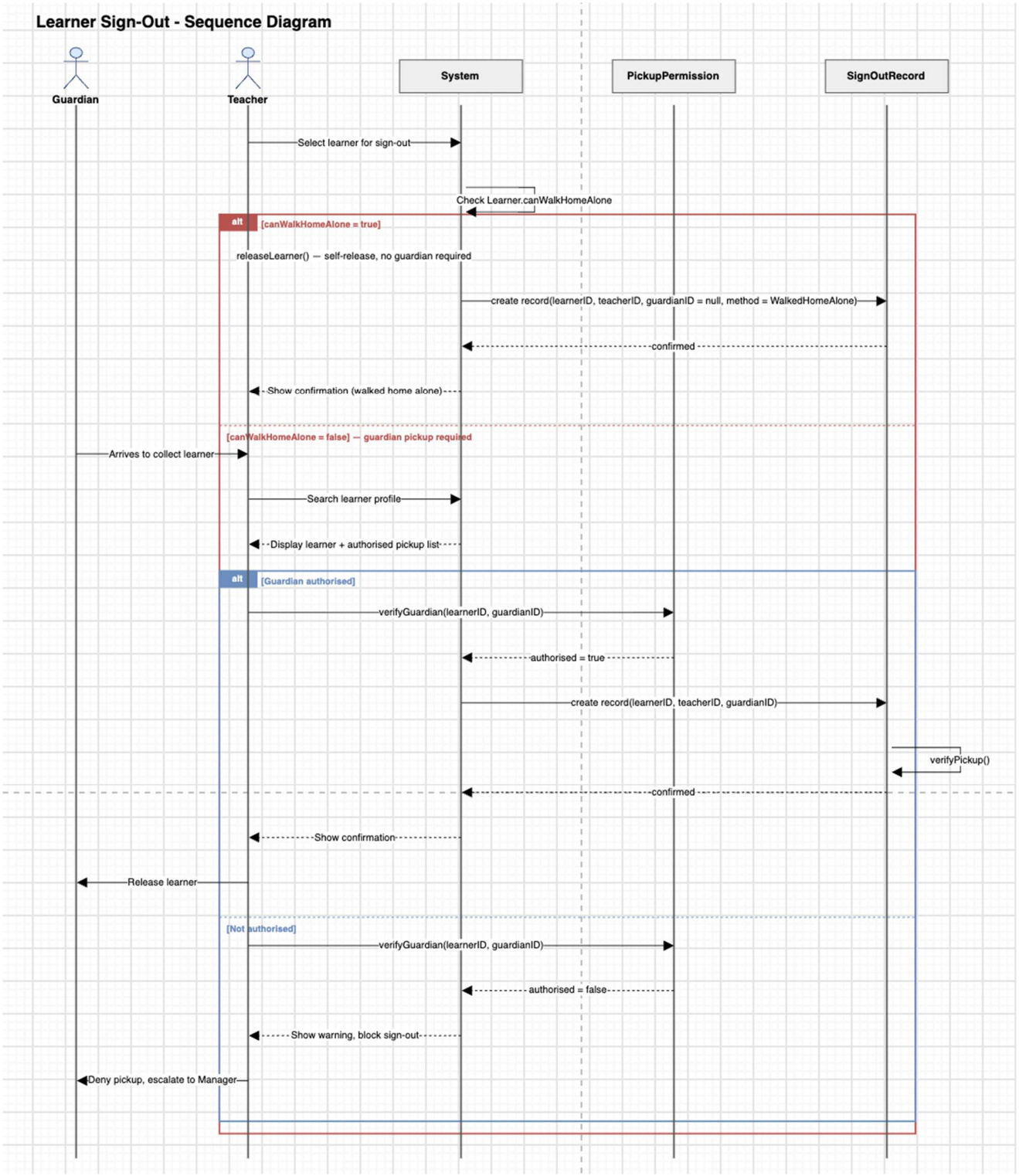


Figure 13: Sequence Diagram for Learner Sign-Out

This sequence diagram titled Learner Sign-Out - Sequence Diagram outlines the technical flow and logic for releasing learners.
Participating Entities:

- Actors: Guardian, Teacher
- System Components: System, PickupPermission, SignOutRecord

Process Flow

1. Initial Setup & Walk Home Alone Check
    - The Teacher selects a learner for sign-out using the System.

- The System internally executes Check Learner.canWalkHomeAlone

2. Condition A: canWalkHomeAlone = true
    - The Teacher sends a releaseLearner() request (self-release, no guardian required) to the System.
    - The System creates a record via SignOutRecord:
      create record(learnerID, teacherID, guardianID = null, method = WalkedHomeAlone).
- SignOutRecord returns a confirmed response to the System.
- The System displays a confirmation ('walked home alone') to the Teacher.

3. Condition B: canWalkHomeAlone = false (Guardian Pickup Required)
    - The Guardian arrives to collect the learner.
    - The Teacher searches the learner profile on the System, which returns and displays the learner along with an authorised pickup list.
    - The process then evaluates the guardian's authorisation status:
        - Case 1: Guardian Authorised
            - The Teacher calls verifyGuardian(learnerID, guardianID) on PickupPermission, which returns authorised = true to the System.
            - The System requests create record(learnerID, teacherID, guardianID) from SignOutRecord.
            - SignOutRecord executes verifyPickup() internally and sends back a confirmed response.
            - The System shows a confirmation to the Teacher, who then releases the learner to the Guardian.
        - Case 2: Not Authorised
            - The Teacher calls verifyGuardian(learnerID, guardianID) on PickupPermission, which returns authorised = false to the System.
            - The System shows a warning and blocks the sign-out.
            - The Teacher denies the pickup and escalates the issue to a Manager.

Deployment and Client Handover Plan

Introduction

The purpose of this deployment and handover plan is to ensure that the Imizamo Yethu Aftercare Management System can be successfully transferred from the student development team to the organisation and maintained after project completion.

The handover includes the deployed application, source code, technical documentation, database information, operational guidelines, and relevant account access. It also ensures that the organisation understands how to use the system, manage users, maintain records, and respond to common technical issues.

Since the application manages information relating to children, particular attention must be given to security, data confidentiality, account ownership, and operational continuity. The organisation should designate an appropriate representative responsible for the system's ongoing administration and maintenance.

Development environment

The development environment is used by team members while implementing and testing application features. Developers run the Express server and React development server locally, connecting to either a local PostgreSQL instance or an isolated development database.

Development databases must remain separate from production data to prevent testing activities from affecting real learner, guardian, attendance, or organisational records. Fictional or anonymised information should be used during testing, and production credentials must not be stored in local development environments.

Staging environment

A dedicated staging environment was considered but was not selected for the initial implementation.

The project's budget constraints and the limitations of Azure App Service Basic influenced this decision. The Basic tier does not support deployment slots, meaning a separate staging App Service would need to be provisioned and maintained.

The development team selected automated testing during pull requests and integration stages, followed by manual quality assurance before production deployment. This approach reduces additional hosting costs and maintenance overhead while providing a degree of protection against introducing defects.

However, automated testing is not a complete replacement for staging. A separate staging environment should be considered in the future if the organisation's budget and technical resources allow it.

Production environment

The production environment is the live, client-facing application hosted on Azure App Service Basic. It is accessible through the organisation's existing registered .org.za domain and secured using a Let's Encrypt SSL certificate.
The production environment contains the live application and its connection to Azure Database for PostgreSQL Flexible Server. It must be treated as the authoritative source of organisational information, and changes must be performed carefully to prevent data loss or service disruption.

Hosting overview

Component	Technology/Platform	Purpose
Frontend	React	User interface for staff, administrators, and parents
Backend	Node.js and Express	Application logic, API requests, authentication, and database operations
Application Hosting	Azure App Service Basic B1 Tier	Hosts the combined frontend and backend deployment
Database	Azure Database for PostgreSQL Flexible Server, Burstable Tier	Stores learner, guardian, attendance, and other records
SSL	Let's Encrypt	Provides HTTPS encryption
Domain	Existing .org.za domain	Provides public application access
Source Code	GitHub	Version control and development history
Deployment	GitHub Actions	Automated testing and deployment

Figure 14: Table Showing The Hosting Overview

The React frontend is built into static files and served by Express. The backend communicates with the separately hosted PostgreSQL database.

The frontend and backend are currently deployed together to reduce deployment complexity and hosting costs. Separating them into independently deployable services may be considered in the future if scalability or maintenance requirements increase.

Deployment, Rollback, and Configuration

Deployment is automated through GitHub Actions and follows the team's branching strategy: each member works on a personal branch and opens a pull request into main. There is no develop branch.
The release process is:

1. Developers implement changes on their personal branches and create pull requests into main.
2. Linting and automated tests, including integration tests against a temporary PostgreSQL database, run against each pull request.
3. Approved pull requests are merged into main.
4. A merge into main triggers GitHub Actions to build the React frontend, bundle it with Express, and deploy the application to Azure App Service.
5. The deployed application is checked to confirm that it remains accessible and functional.

1. Deployment Configuration

Sensitive configuration values, including database connection information, JWT signing secrets, and API keys, must not be committed to the GitHub repository.
Production environment variables are maintained through Azure App Service Application Settings. GitHub Actions authenticates to Azure using credentials stored in GitHub Secrets.
The organisation must ensure that access to Azure and GitHub is restricted to authorised individuals. Credentials should be transferred securely and changed when ownership changes or unauthorised access is suspected.

2. Deployment Downtime

Azure App Service Basic does not support deployment slots. Consequently, production deployments may briefly interrupt access while the application restarts.
This limitation was accepted as a cost-conscious decision for the initial implementation. Significant updates should preferably be scheduled outside the aftercare centre's busiest operational hours.

3. Rollback and Recovery

If a production deployment introduces a critical issue, the last known stable application version should be redeployed through the approved GitHub Actions workflow.
The process involves identifying the problematic deployment, selecting the previous stable commit, running the required tests, redeploying, and verifying that the application functions correctly.
Application rollback does not automatically reverse database changes. If a deployment modifies the database schema or records, the technical maintainer must separately assess whether database recovery or corrective action is required.

Client Handover and Responsibilities

1. Handover Objectives

The handover ensures that the organisation receives the application and understands how to operate and maintain it.
The development team should provide:

- The deployed application and source code repository.
- System architecture and database documentation.
- OpenAPI documentation for the backend API.
- User manuals and administrator guidelines.
- Deployment and environment configuration instructions.
- Backup, recovery, and troubleshooting guidance.
- Testing documentation and known limitations.
- Training for relevant staff members.

The organisation must designate a representative responsible for ongoing administration, account ownership, maintenance, and technical support arrangements.

2. Handover Responsibilities

Responsibility	**Development Team**	**Aftercare Organisation**
Application	Deliver and demonstrate the system	Review functionality and accept handover
Source Code	Provide repository access and documentation	Maintain access for authorised developers
Hosting	Explain Azure deployment and configuration	Maintain Azure account and billing responsibility
Database	Explain schema, access, and recovery requirements	Maintain database access and coverage
User Training	Demonstrate application functionality	Ensure staff receive training
Security	Document security measures and limitations	Maintain account security and protect information

Maintenance	Explain technical maintenance procedures	Designate an ongoing technical maintainer

3. Account Ownership

The organisation should confirm ownership and authorised access to the following:

- **Azure:** App Service, PostgreSQL server, subscription, billing, and application configuration.
- **GitHub:** Repository, deployment workflow, and administrator permissions.
- **Domain:** Existing `.org.za` registration and DNS configuration.
- **SSL:** Certificate configuration and renewal responsibilities.

Access should be assigned through appropriate permissions rather than shared administrator passwords. The PostgreSQL database must only be accessible to authorised technical personnel. Teachers and parents should interact with the application rather than accessing the database directly.

Operational and Maintenance Guidelines

1. User Responsibilities

Users must access the application using their assigned accounts and should not share login credentials. Teachers should verify the correct learner before recording attendance, observations, or sign-out information. Guardian verification procedures must follow the centre's established safeguarding policies. The Manager adds, suspends and restores teacher accounts, and the Master Administrator manages Manager and Administrator accounts and resets forgotten staff passwords. They are responsible for reviewing access permissions, and assisting users with operational problems.
Parents and guardians should only access information and functionality available to their assigned accounts.
If the application becomes unavailable, staff should follow the centre's approved temporary attendance and sign-out procedures. Any records created during the interruption should be reconciled with the system once service is restored.

2. Database Management

The system uses Azure Database for PostgreSQL Flexible Server with the Burstable compute tier.
The organisation must verify that the database remains covered by its Azure nonprofit credit arrangement. Coverage depends on eligibility, available credits, the specific database configuration, and annual renewal requirements.
The organisation should:

- Monitor database availability and Azure credit consumption.
- Complete required annual nonprofit registration or renewal.
- Review database costs before coverage expires.
- Maintain appropriate backup and recovery procedures.
- Restrict database access to authorised individuals.

Azure provides managed backup capabilities for PostgreSQL Flexible Server, but the organisation must verify the actual backup configuration and retention period.

Backup and recovery documentation should explain how backups are monitored, who is responsible for restoration, and how the organisation should respond to accidental data loss or corruption.

Database schema changes must be reviewed, documented, and tested before being applied to production. Schema changes are managed with node-pg-migrate: the migration files in the repository run automatically when the application starts, which keeps every environment consistent.

3. Security and Data Protection

The application includes security measures such as bcrypt password hashing, JWT-based authentication, HTTP-only cookies where configured, role-based access control, server-side validation, Helmet security

headers, rate limiting, and HTTPS. These measures must be supported by responsible organisational practices.

The organisation should:

- Remove unnecessary user access when staff members leave.
- Review administrative permissions periodically.
- Avoid shared accounts and credentials.
- Protect learner and guardian information.
- Avoid storing sensitive information on personal devices without authorisation.
- Report suspected security incidents immediately.
- Change compromised credentials and follow appropriate incident procedures.

Learner information should only be accessed for legitimate organisational purposes and in accordance with applicable data-protection requirements.

6. Troubleshooting and Support

The organisation should designate a primary contact responsible for receiving and escalating issues.

Issue	Recommended Action
Application unavailable	Check Azure App Service availability and deployment records.
Login unsuccessful	Verify credentials and contact the administrator.
Database connection failure	Check PostgreSQL availability and configuration through authorised access.
Application error after deployment	Review deployment logs and restore the last stable version if necessary.
SSL warning	Check certificate status and domain configuration.
Incorrect attendance record	Verify the learner and follow the approved correction procedure.
Application slow	Review hosting resources, database performance, and application usage.
Azure credits exhausted	Review billing and arrange appropriate funding or service changes.

The support process should distinguish between user issues, administrative issues, technical failures, and security incidents.

The student development team may provide initial handover assistance according to the agreed project arrangements. Ongoing technical support after project completion should not be assumed unless formally agreed.

Maintenance and Future Improvements

The organisation should establish a maintenance schedule to support continued operation.

Frequency	Recommended Activity	Responsible Party
Weekly	Check application availability and review reported issues	Administrator
Monthly	Review user accounts, backup status, and Azure credit consumption	Administrator / Technical Maintainer
Quarterly	Review permissions, security practices, and system performance	Organisation / Technical Maintainer
Annually	Complete nonprofit renewal and review domain, hosting, and operational costs	Organisation
As required	Apply approved updates, security fixes, and enhancements	Technical Maintainer

The following improvements may be considered as the system develops:

- Dedicated staging environment: Allows updates to be tested before production deployment.
- Automated end-to-end testing: Playwright could verify workflows such as login, attendance, guardian verification, sign-out, and report generation.
- Post-deployment health checks: Confirm that the application remains accessible after releases.
- Improved monitoring: Track application errors, database availability, performance, and Azure credit usage.
- Independent frontend and backend deployment: Consider separating services if future requirements justify the additional complexity.

These improvements should be prioritised according to the organisation's budget, operational needs, and technical resources.

Client Training and Handover Acceptance

1. Training Procedure

A practical training session should be conducted with the relevant staff members.
The session should demonstrate:

- Logging in and navigating the application.
- Understanding user roles and permissions.
- Managing learner records.
- Recording attendance and observations.
- Completing guardian verification and sign-out procedures.
- Generating and accessing reports.
- Reporting common issues and understanding account security.

Training should use clear instructions, screenshots, and practical demonstrations suited to the staff's technical experience.

2. Handover Acceptance Checklist

Handover Item	Confirmed
Application deployed and accessible	☐
Source code repository access provided	☐
System and database documentation provided	☐
OpenAPI documentation provided	☐
User and administrator manuals provided	☐
Deployment and configuration guides provided	☐
Backup and recovery procedures explained	☐
Azure, database, domain, and account responsibilities reviewed	☐
Security and maintenance responsibilities explained	☐
User training completed	☐
Known limitations and future recommendations discussed	☐
Client acceptance recorded	☐

Handover Acceptance Record

Organisation: Imizamo Yethu Aftercare Centre

System: Aftercare Management System

Date of Handover: _____

Development Team Representative: _____

Client Representative: _____

Outstanding Issues or Recommendations:

Client Representative Signature: _____

Development Team Representative Signature: _____

Conclusion

The deployment and handover plan provides a structured approach to transferring the Aftercare Management System from the student development team to Imizamo Yethu Aftercare Centre.

The current solution combines React and Express on Azure App Service, with PostgreSQL Flexible Server providing database functionality. GitHub Actions supports automated deployment, while the organisation's existing .org.za domain and SSL certificate provide secure application access.

The cost-conscious deployment approach is appropriate for the initial implementation, although the organisation should review its hosting, testing, monitoring, and maintenance arrangements as the system develops.

The handover aims to ensure that the organisation receives not only a functional application, but also the documentation, training, and operational knowledge required to maintain it responsibly after the completion of the academic project.

Data Schema

Choice of Technology

1. Purpose of the Document

The purpose of this document is to evaluate and select the technologies that will be used to develop, deploy and maintain the Imizamo Yethu Aftercare Management System. The technologies will be evaluated according to the functional and non-functional requirements of the system, as well as the resources and technical environment available at the aftercare.
The selected technology stack should support a mobile-responsive web application, secure learner information, multiple user roles, reporting, analytics and future scalability.

---

2. Technology Requirements

2.1 Application Requirements

Requirement	Technology Consideration
Mobile-responsive web application	Must work effectively on smartphones, tablets and computers
Simple user interface	Must support intuitive interfaces suitable for users with limited technical experience
Attendance management	Must support fast reading and writing of attendance records
Learner profiles	Requires structured and persistent data storage
Parent portal	Requires secure authentication and controlled access
Multiple user roles	Must support role-based authorisation
Digital sign-out	Must securely record collection information and history
Reporting	Must support PDF generation
Dashboard	Must support charts, statistics and data summaries
Sensitive learner information	Requires appropriate security and access controls
Future growth	Should support expansion and additional features

2.2 Environmental Constraints

Available devices: Teachers primarily have access to smartphones, while the manager has access to a laptop.
Internet/network: The aftercare has reliable existing Wi-Fi.
Technical literacy: The system must be suitable for teachers with limited technical experience.
Budget: An exact budget was not confirmed, however costs are to be kept to a minimum.
Development team: Azure Services, Visual Studio, C#, JavaScript, Java, HTML, SQL, Oracle, MongoDB, Android Studio, Visual Studio Code.

---

3. Technology Evaluation Criteria

Criteria	Description
Suitability	How well the technology supports the system requirements
Mobile responsiveness	Ability to provide an effective experience on smartphones
Security	Support for authentication, authorisation and secure data handling
Performance	Ability to provide fast system responses
Scalability	Ability to support increased users, data and future functionality
Maintainability	Ease of updating, debugging and maintaining the system
Cost	Development, hosting, licensing and database costs
Development time	Whether the technology allows completion within the available timeframe
Team experience	Development team's familiarity with the technology
Community/support	Availability of documentation, libraries and developer support
Integration	Ability to integrate with databases, PDF generation, charts and other functionality

4. Application Architecture

4.1 Architecture Options Considered

- Option 1: Mobile application
- Option 2: Responsive web application
- Option 3: Progressive Web Application (PWA)

Criteria	Mobile App	Responsive Web App	PWA
Smartphone access	Excellent (native)	Good (via browser)	Good (installable, app-like)
Laptop access	Poor (not designed for desktop)	Excellent	Good
Installation required	Yes (app store download)	No	Optional (add to home screen)
Development complexity	High (native/cross-platform build + app store approval)	Low-Medium (single codebase)	Medium (service workers, manifest, offline caching)
Cost	Higher (separate builds, possible store fees)	Lower (one codebase, standard web hosting)	Medium
Maintainability	Harder (OS updates, store resubmissions)	Easier (single codebase, instant updates)	Medium
Suitability for project	Low	High	Medium (viable future enhancement)

4.2 Selected Architecture

Technology/Approach: Responsive Web Application
Justification:
A responsive web application was selected because it allows a single codebase to serve both teachers (smartphones) and the manager/administrator (laptop/desktop) without the cost and complexity of building and maintaining separate native mobile and desktop apps. It requires no app store installation or approval process, which suits teachers with limited technical experience and removes barriers to adoption. It works over the aftercare's existing Wi-Fi through any standard browser, keeping hosting and development costs low, which is important given the non-profit's limited budget. A native mobile app (Option 1) was rejected due to higher development complexity, cost and the need for the manager to also access the system on a laptop. A PWA (Option 3) was considered for its offline/installable capabilities but was not selected for the MVP as offline functionality is not a stated client requirement and would add unnecessary development complexity within the project timeline; it remains a viable future enhancement.

---

5. Frontend Technology

5.1 Requirements

- Provide a mobile-responsive interface
- Work on common smartphone and desktop browsers
- Be simple for teachers to use
- Provide large/simple controls where appropriate
- Support dashboards, forms, attendance and learner profiles

5.2 Technologies Considered

Option 1: React (with Tailwind CSS)

Advantages: Strong component reusability, large ecosystem and community support, excellent support for responsive design via Tailwind CSS and the development team already has direct experience with React and JavaScript.
Disadvantages: Slightly steeper learning curve than plain HTML/CSS/JS for very simple pages; requires a build/bundling step.

Option 2: Vue.js

Advantages: Gentle learning curve, good documentation, built-in reactivity, capable of responsive design.
Disadvantages: Smaller ecosystem than React, and no prior team experience, which increases development time and risk given the project deadline.

Option 3: Plain HTML, CSS and JavaScript

Advantages: No build tooling required, very simple to get started, full control over markup.
Disadvantages: No component structure, becomes difficult to maintain and scale as the number of screens grows (dashboards, forms, attendance, learner profiles), more manual work to achieve consistent responsive behaviour and accessibility.

Criteria	Option 1	Option 2	Option 3
Mobile responsiveness	High (with Tailwind CSS)	High	Possible but manual
Ease of development	Medium (some learning curve, but team has experience)	Medium	Easy for small scope, hard to scale
Performance	Good	Good	Good (raw, but no structure)
Maintainability	High (component-based)	High (component-based)	Low as app grows
Team experience	High (prior team experience)	Low (no prior experience)	Medium
Overall suitability	High	Medium	Low

5.3 Selected Technology

Technology: React (with Tailwind CSS for styling)
Justification: React was selected because it directly supports the requirement for a mobile-responsive interface with reusable components (attendance forms, learner profile cards, dashboards), the team already has practical experience with it which reduces development time and risk, and Tailwind CSS makes it straightforward to build the large, simple touch-friendly controls needed for teachers with limited technical experience. React also integrates cleanly with a Node.js/Express backend, since both share the JavaScript ecosystem, simplifying tooling and reducing context-switching for the development team.

---

6. Backend Technology

6.1 Requirements

The backend needs to support authentication, role-based permissions, attendance, learner records, parent access, digital sign-out, reporting and historical data.

6.2 Technologies Considered

- Option 1: Node.js with Express.js
- Option 2: Django (Python)
- Option 3: ASP.NET Core (C#)

Criteria	Option 1	Option 2	Option 3
Security	High (full control over middleware-based auth/RBAC)	High (built-in Django auth system)	High (built-in ASP.NET Identity)
Scalability	Good (widely used in production at scale)	Good	Good
Integration	Excellent (same language as React frontend; shared JSON handling)	Good, but separate language from frontend	Good, but separate language from frontend
Development time	Fast (team JavaScript experience, lightweight framework)	Medium (no prior team experience with Python/Django)	Medium (some C# experience, but ASP.NET has more boilerplate)
Team experience	High (prior experience)	Low	Medium
Maintainability	High (single language across stack)	Medium (context-switching between Python and JavaScript)	Medium (context-switching between C# and JavaScript)
Overall suitability	High	Medium	Medium

6.3 Selected Technology

Technology: Node.js with Express.js
Justification: Node.js with Express.js was selected because it shares the same language (JavaScript) as the React frontend, reducing complexity and allowing the team to move faster given prior team experience with JavaScript. Express provides fine-grained control over middleware, which is used to implement role-based authorization as a Chain of Responsibility pattern (see Data Schema, Architecture Artifacts): every request passes through logging, authentication and role-authorisation checks before reaching a route handler. This gives the team full control over exactly how each of the five user roles' permissions are enforced, which is important given the system's varied access levels (Master Administrator, Administrator, Manager, Teacher, Parent/Guardian). Django (Option 2) was considered but would require the team to learn Python for this project with no prior experience, increasing risk. ASP.NET Core (Option 3) was considered given some team members' C# experience, but Node.js/Express keeps the whole stack within a single language, which was judged more efficient for a small team on a fixed deadline.

---

7. Database Technology

7.1 Data Requirements

- Users and user roles
- Learners
- Parents/guardians
- Classes
- Attendance records
- Pickup permissions
- Sign-out records
- Learner observations
- Development records
- Historical records

7.2 Technologies Considered

- Option 1: PostgreSQL (Azure Database for PostgreSQL)

- Option 2: MongoDB (NoSQL)
- Option 3: MySQL (self-managed)

7.3 Selected Database

Technology: PostgreSQL (Azure Database for PostgreSQL: Flexible Server, Burstable tier)
Justification: PostgreSQL is a relational database that naturally models the clearly related entities in this system, including users, learners, parents/guardians, classes, attendance records, pickup permissions, sign-out records, learner observations and historical data. Using Azure's managed Flexible Server (Burstable tier) avoids the cost and operational overhead of self-managing a MySQL server (Option 3), while providing automatic backups, patching and SSL-enforced connections by default. A NoSQL option such as MongoDB (Option 2) was considered but was not selected, as the strongly relational nature of this data; particularly the need to reliably enforce which guardians may collect which learner; is a better fit for PostgreSQL's relational integrity constraints (foreign keys, joins) than a document-based model.

---

8. Hosting and Deployment

8.1 Requirements

- Internet accessibility
- Reliability during aftercare hours
- Cost
- Security
- Backups
- Scalability
- Ease of deployment
- Integration with selected backend/database

8.2 Options Considered

- Option 1: Microsoft Azure (App Service + Azure Database for PostgreSQL)
- Option 2: Vercel (frontend) + Render (backend/database)
- Option 3: Self-managed VPS (e.g. DigitalOcean droplet)

8.3 Selected Hosting Platform

Technology: Microsoft Azure: Azure App Service (Basic tier) hosting the Node.js/Express application (serving the built React frontend from the same instance), with Azure Database for PostgreSQL (Flexible Server, Burstable tier) for the database
Justification: Hosting both the application and database within Azure keeps the system within a single, consistently managed cloud provider, simplifying billing, monitoring and support. Azure App Service provides automatic HTTPS via a Let's Encrypt certificate on the aftercare's custom domain, and Azure Database for PostgreSQL provides automatic backups and enforces SSL on all connections by default. The PostgreSQL server's built-in firewall restricts inbound connections to Azure services and the App Service's specific outbound IP address, meaning the database has no public exposure even without full VNet isolation (which was considered but not selected, as it requires the Standard App Service tier and above and was not justified against the added monthly cost for a project at this scale). A self-managed VPS (Option 3) was rejected, as it would place server maintenance and security patching responsibility on the client after handover, which is not appropriate given the aftercare's limited technical capacity. A split Vercel/Render setup (Option 2) was considered but was not selected in favour of keeping the whole backend and database within one provider for simpler ongoing management.

---

9. Authentication and Security

9.1 Security Requirements

- Secure login
- Role-based access
- Protection of learner information
- Protection of guardian information
- Secure parent access
- Restricted administrative functions
- Secure communication between browser and server
- Appropriate password handling

9.2 Selected Security Technologies/Approach

Authentication: JSON Web Tokens (JWT), issued by the Express backend after verifying credentials against bcrypt-hashed and salted passwords stored in PostgreSQL
Authorisation: Role-based access control enforced through Express middleware (Chain of Responsibility pattern); every request passes through authentication and role-authorisation checks before reaching a route handler.

Data protection: HTTPS/TLS between client and App Service (Let's Encrypt certificate); SSL-encrypted connections between App Service and PostgreSQL; database firewall restricting access to Azure services and the App Service's outbound IP only; parameterised queries (via the Repository pattern) to prevent SQL injection

Other: Server-side input validation (type, length and format checks), React's automatic output escaping and a Helmet Content Security Policy header to mitigate cross-site scripting (XSS); JWT stored in an httpOnly, Secure, SameSite=Strict cookie to reduce XSS token theft and CSRF exposure; login endpoint rate-limited to slow brute-force attempts

Justification: This approach directly addresses the system's security requirements. Secure login and password handling are provided through bcrypt hashing and JWT-based authentication. Role-based access to learner and guardian information is enforced centrally through Express's middleware chain, meaning every one of the system's five user roles has their permissions checked consistently at a single, testable layer rather than scattered across individual route handlers. All communication is encrypted, between browser and server and between server and database, and the database's public endpoint only accepts connections allowed by its firewall. Output escaping and input validation address the XSS requirement highlighted in the module rubric, and storing the JWT in an httpOnly cookie mitigates both XSS-based token theft and CSRF in a single decision.

---

10. Reporting and Analytics Technology

The system requires PDF donor/board reports, attendance statistics, learner development summaries, charts and analytics.

10.1 Technologies Considered

PDF generation: @react-pdf/renderer (client/server-side PDF generation library for React)
Charts/analytics: Recharts (React charting library)

10.2 Selected Technologies

PDF: @react-pdf/renderer with html-to-image
Charts: Recharts
Justification: @react-pdf/renderer integrates directly with the chosen React frontend, and is combined with html-to-image, which captures each report card as it appears on screen, so the PDF matches the page exactly, charts included, without a separate reporting technology stack. Recharts is a lightweight, React-native charting library well suited to the attendance statistics, learner development summaries and dashboard analytics required, and it has good documentation and community support for a team working under a fixed deadline.

---

11. Development and Collaboration Tools

11.1 Version Control

Selected Technology: GitHub
Justification: GitHub supports the module's required branching strategy (personal branches with pull requests into main, as documented in the DevOps section) and enables the whole team to collaborate on the same codebase with a clear history of changes. It integrates directly with Azure for automated deployment and supports GitHub Actions for the CI/CD pipeline and automated testing required for Task 2.

11.2 Development Environment / IDE

Options considered: Visual Studio Code, Visual Studio
Selected technology: Visual Studio Code
Justification: Visual Studio Code is free, lightweight, cross-platform (so all group members can use it regardless of operating system) and has strong extension support for React, Node.js/Express and PostgreSQL, which are the core technologies selected above.

11.3 Testing Tools

Selected technology/technologies: Vitest, React Testing Library and jest-axe (frontend); Jest and Supertest (backend API and integration testing against PostgreSQL)
Purpose: Automated unit and component testing of the React frontend and automated endpoint testing of the Express API, integrated into the GitHub Actions CI/CD pipeline so tests run automatically on code changes.
Justification: Vitest runs the frontend tests in the same Vite tool that builds the React app and shares Jest's API, and Jest runs the backend tests, so the team uses one testing style across the whole JavaScript stack. Supertest allows Express API endpoints to be tested directly without needing a live server running, and both integrate directly with GitHub Actions to satisfy the module’s requirement for automated testing as part of the development pipeline.

---

12. Final Technology Stack

Component	Selected Technology	Purpose	Main Reason for Selection
Architecture	Responsive Web Application	Overall application structure	Single codebase for phones + laptops, no install/app-store friction, low cost
Frontend	React (with Tailwind CSS)	User interface	Team experience, component reuse, strong responsive support
Backend	Node.js with Express.js	Business logic/API	Same language as frontend, fine-grained middleware control for RBAC, team experience
Database	PostgreSQL (Azure Database for PostgreSQL)	Data storage	Relational structure fits learner/attendance/ guardian data; managed backups and SSL by default
Authentication	JWT + bcrypt password hashing	User login/security	Secure login with role embedded in token; stored in httpOnly cookie to reduce XSS/CSRF risk
Hosting	Microsoft Azure (App Service + Azure Database for PostgreSQL)	Deployment	Single cloud provider, automatic HTTPS, managed backups, firewall-restricted database access
PDF Generation	@react-pdf/renderer with html-to-image	Donor/board reports	Integrates directly with React frontend
Charts	Recharts	Dashboard analytics	Lightweight, React-native charting library
Version Control	GitHub	Code management and collaboration	Required by module; supports branching + GitHub Actions CI/CD
Testing	Vitest, React Testing Library, jest-axe, Jest, Supertest	System testing	Vitest for the React app; Jest and Supertest for the Express API; one testing style across the JavaScript stack

13. Conclusion

The selected technology stack was chosen based on the functional requirements, non-functional requirements, stakeholder needs, project constraints and capabilities of the development team. The technologies selected should enable the development of a secure, responsive and maintainable web application that can be accessed easily by teachers using smartphones while providing more detailed functionality to management and other authorised users.

Entity Relationship Diagram

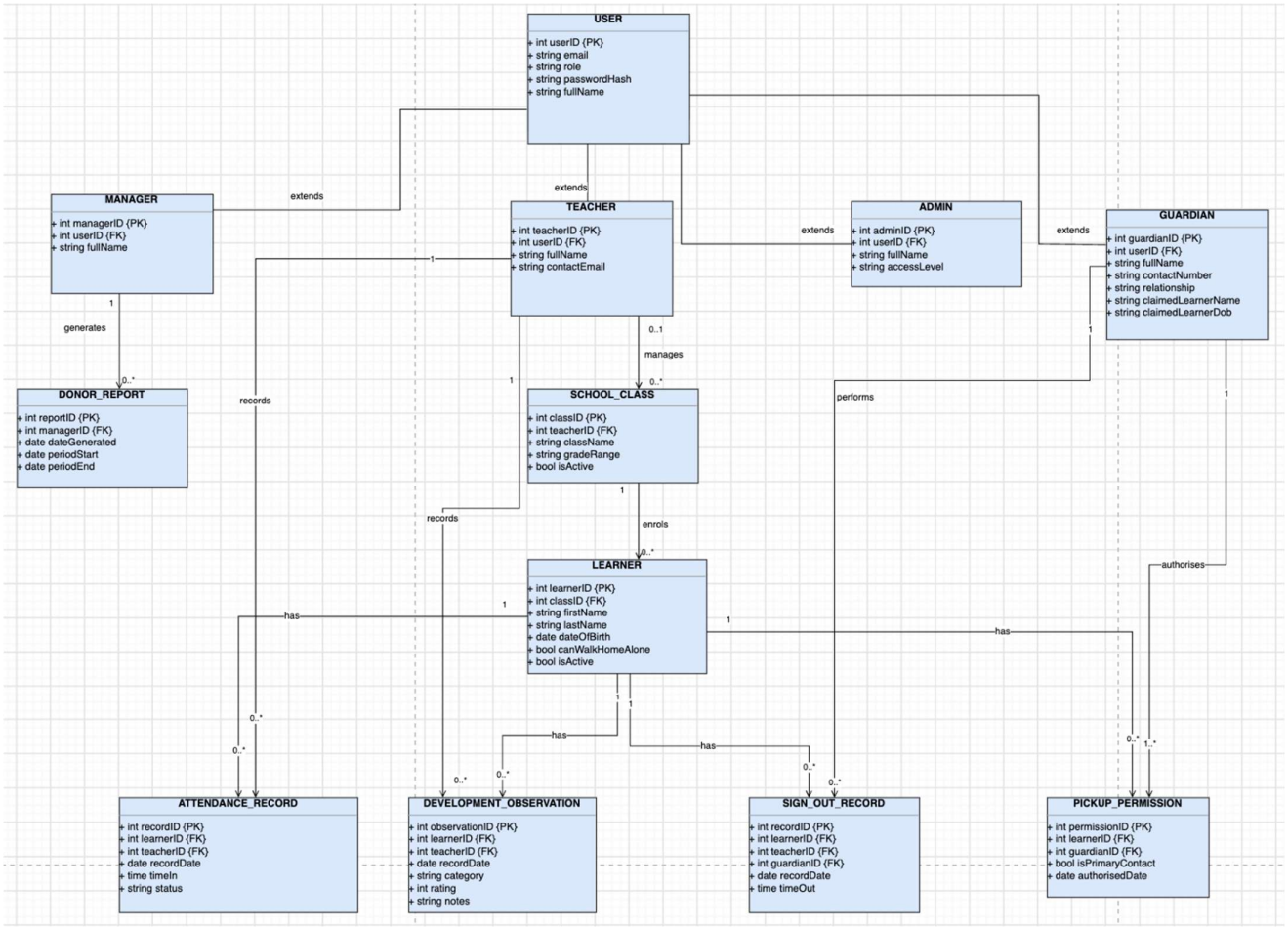


Figure 15: Entity Relationship Diagram

This ERD class diagram outlines the OOD and the relational data structure for the Imizamo Yethu Aftercare Management System. The schema is structured to optimize the daily attendance, ensure children's safety for pick-up procedures, log development observations and support the board and donor reporting.

User Authentication and Role Hierarchy

The domain model utilizes generalization to centralize authentication logic in a parent entity (USER entity), which then gets extended into four sub operational roles.

USER (Base Entity)

- Attributes: userID (PK), email, role, passwordHash, fullName
- This serves as the base account table that will store common authentication credentials and identity details across all user types.

ADMIN

- Attributes: adminID (PK), userID (FK), fullName, accessLevel
- This serves as a USER extension for board-level access (e.g system configuration, term setups and the organizational overview)
- Relationships: Performs configuration and administrative functions across the application.

MANAGER

- Attributes: managerID(PK), userID (FK), fullName
- This extends USER to manage aftercare operations across all the classes

- Relationships: Generates 0..* DONOR_REPORT entities.

TEACHER

- Attributes: teacherID(PK), userID(FK), fullName, contactEmail
- This serves to extend USER for daily class management and mobile data logging
- Relationships:
  - Manages 0..* SCHOOL_CLASS entities
  - Records 0..* ATTENDANCE_RECORD entities
  - Records 0..* DEVELOPMENT_OBSERVATION entities
  - Performs 0..* SIGN_OUT_RECORD actions

GUARDIAN

- Attributes: guardian{PK} userID(FK), fullName, contactNumber, relationship, claimedLearnerName, claimedLearnerDob
- Extends USER to allows parent or guardian to access child progression and set pickup authorisations
- Relationship:
  - Authorises 0..* SIGN_OUT_RECORD entities
  - Participates in 0..* SIGN_OUT_RECORD entries upon collecting a child

---

Classroom Structure and Learner Profiles

To accommodate quick navigation on smartphones, learners are linked directly to their respective classes

SCHOOL_CLASS

- Attributes: classID(PK), teacherID(FK), className, gradeRange, isActive
- It represents an aftercare classroom group
- Relationship: Managed by 1 TEACHER; enrolls 0..* LEARNER entities

LEARNER

- Attributes: learnerID(PK), classID(FK), firstName, lastName, dateOfBirth, canWalkHomeAlone, isActive
- Serves as the central entity representing an enrolled child in the aftercare program
- Relationships:
  - Enrolled in SCHOOL_CLASS
  - Has 0..* ATTENDANCE_RECORD entities
  - Has 0..* DEVELOPMENT_OBSERVATION entities
  - Has 0..* SIGN_OUT_RECORD entries
  - Has 0..* PICKUP_PERMISSION authorisations

Operations and Safety Records

Three main transactions entities handle day-to-day records logged by teachers:

ATTENDANCE_RECORD

- Attributes: recordID(PK), learnerID(FK), teacherID(FK), dateRecordDate, timeTimeIn, status
- This replaces their current paper registers with digital check-ins
- Relationships: Multi-to-one junctions linking 1 LEARNER and 1 TEACHER per record

DEVELOPMENT_OBSERVATION

- Attributes: observationID(PK), learnerID(FK), teacherID(FK), dateRecordDate, category, rating, notes

- This tracks qualitative milestones progress, behavior and well being overtime
- Relationships: Links 1 LEARNER and 1 TEACHER

SIGN_OUT_RECORD

- Attributes:recordID(PK), learnerID(FK), teacherID(FK), guardianID(FK) dateRecordDate, timeTimeOut
- This serves to verify learner collection at the end of the day to ensure children safety
- Relationships: Links 1 LEARNER, 1 TEACHER, and 1 GUARDIAN to form an audit log

---

Verification and Reporting Modules

PICKUP_PERMISSION

- Attributes: permissionID(PK), learnerID(FK), guardian(FK), isPrimaryContact, dateAuthorisedDates
- This stores authorised collection contacts for identity verification prior to signing out
- Relationships: Connects 1 GUARDIAN to 1 LEARNER

DONOR_REPORT

- Attributes: reportID(PK), managerID(FK), dateGenerated, periodStart, periodEnd
- This serves as storing metadata for any exported PDF reports on attendance trends and progress metrics the board members and external donors
- Relationships: Generated by 1 MANAGER

Schema Data Summary

Entity / Class	Primary Key	Foreign Key(s)	Function / Role
USER	userID	*None*	Parent entity for authentication credentials
ADMIN	adminID	userID	Board/system administration access
MANAGER	managerID	userID	Program operational management
TEACHER	teacherID	userID	Daily class management and data entry

GUARDIAN	guardianID	userID	Parent access and child pickup permissions
SCHOOL_CLASS	classID	teacherID	Classroom grouping mechanism
LEARNER	learnerID	classID	Child profile and safety settings
ATTENDANCE_RECORD	recordID	learnerID, teacherID	Daily digital attendance log
DEVELOPMENT_OBSERVATION	observationID	learnerID, teacherID	Child progress and wellbeing notes
SIGN_OUT_RECORD	recordID	learnerID, teacherID, guardianID	Departure verification audit trail
PICKUP_PERMISSION	permissionID	learnerID, guardianID	Authorised pickup registry
DONOR_REPORT	reportID	managerID	Executive report generation metadata

Architecture

Design Patterns

Design patterns are used at the code level to solve implementation problems. The two out of three patterns that are going to be used are illustrated with UML object diagrams. The third is explained in text only, for reasons mentioned below.

---

1. Repository Pattern

This pattern will be used in the data access layer to abstract PostgreSQL queries away from business logic. Services call into repository objects (e.g. learnerRepository.findById(learnerId)) rather than writing raw SQL directly, and controllers only call services. This method decouples the data layer from business logic and makes it easy to unit test with mock data. The UML diagram below shows this in practice: attendanceCtrl, an instance of AttendanceController, links to learnerRepo, an instance of LearnerRepository, which in turn links to pgPool, an instance of PgPool. In the code, the controller reaches the repository through a service (see the System Architecture Diagram) and never touches the raw SQL or connection pool directly.

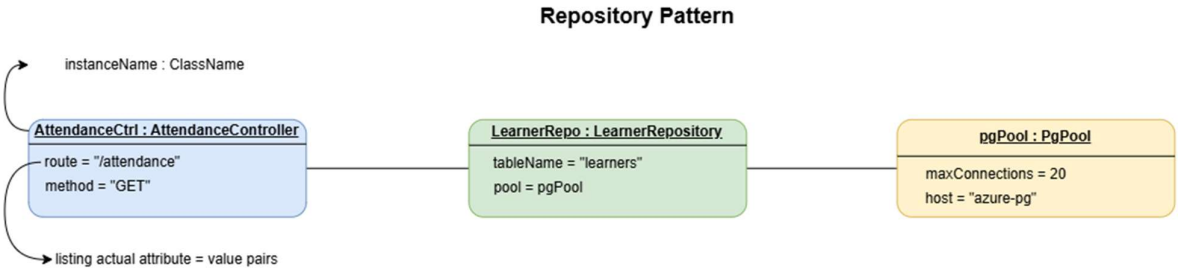


Figure 16: Repository Pattern

---

2. Singleton Pattern

This pattern will be used in the PostgreSQL connection pool (PgPool). A single instance will be made at the application startup and will then be reused across the whole system. The singleton pattern negates the need for each part of the codebase to open its own connection on start up, which would risk exhausting PostgreSQL's connection limit under concurrent load from all five user roles. The UML object diagram below illustrates this concept directly with five separate repository instances (learnerRepo, attendanceRepo, observerRepo, reportRepo and signoutRepo). Each repository holds an identical pool = pgPool attribute value, all (converging) on the same single pgPool object instead of each creating their own.

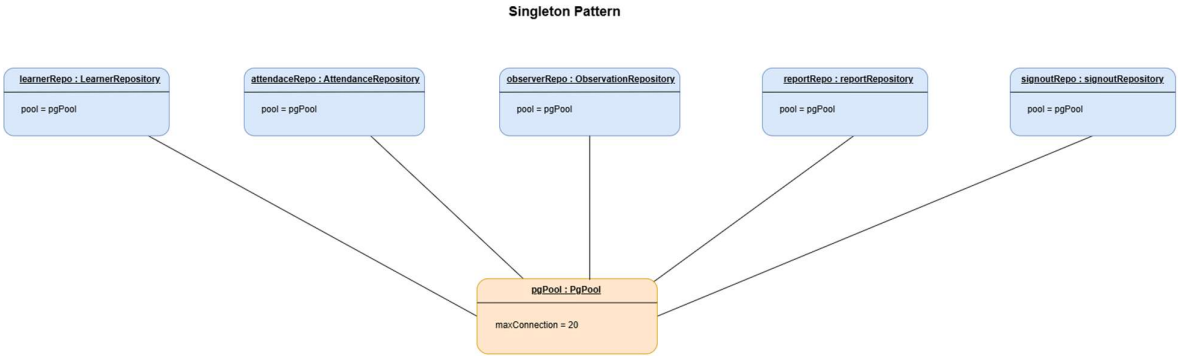


Figure 17: Singleton Pattern

---

3. Middleware (Chain of Responsibility)

By using Express as a middle ware stack it handles the specific tasks such as - request logging, authentication, and role-based authorization as independent functions. They each handle one concern at a time before passing control to the next task. For example, if a user makes a request to the Manage Learners route but only has parent role access, the request will pass through logging and authentication middleware, however it will be stopped by the role-authorization middleware before it ever reaches the route handler and it will return a 403 Forbidden response. This pattern is not represented with a UML object diagram as Chain of Responsibility is a sequence of behaviour over time. Therefore, this pattern is expressed below in a sequence diagram.

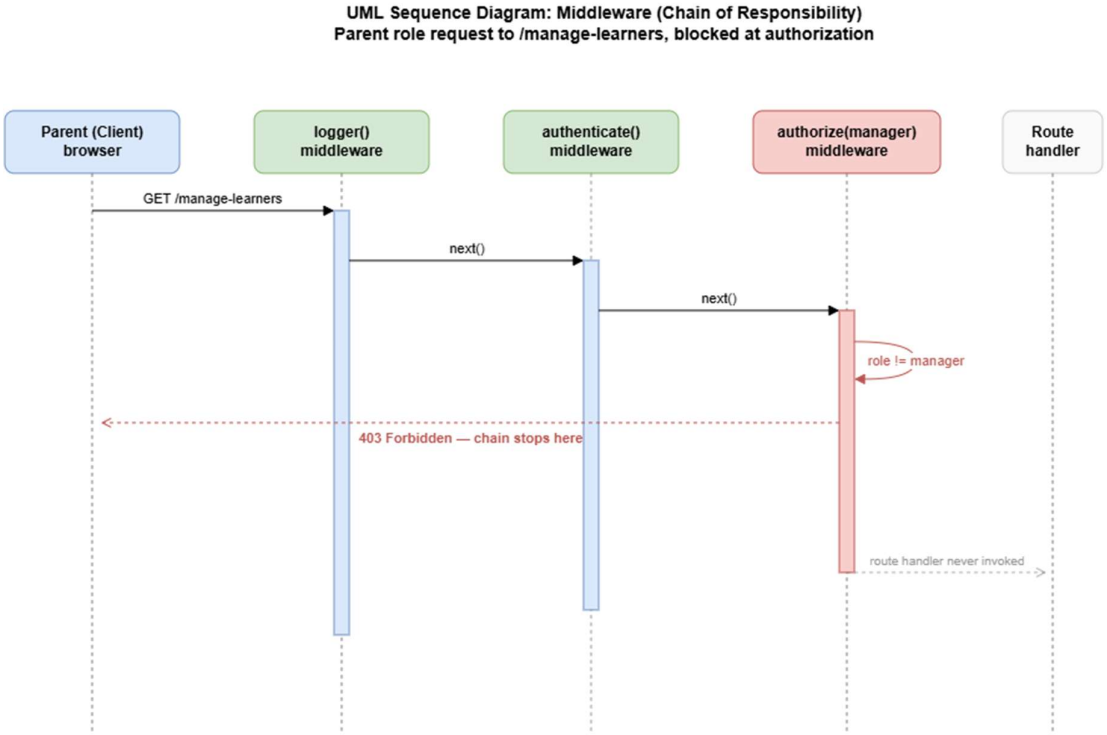


Figure 18: UML Sequence Diagram of Middleware (Chain of Responsibility)

System Architecture Diagram

This diagram shows the flow of a request from the client to the database. The React Client communicates with the backend over the HTTPS using REST conventions such as Client-Server and RESTful API patterns. The dashed border shows the Azure App Service instance, which hosts the entire backend as one deployable unit, using a Monolithic pattern. Inside this dashed border, the backend is split into three layers - Routes/Controllers, Services, and Repositories. This reflects the order a request passes through the system, using a Layered pattern. The repositories layer only connects to PostgreSQL consistently with the Repository design pattern documented above. This diagram expresses all four architecture patterns as complementary views of the same system, rather than as competing alternatives.

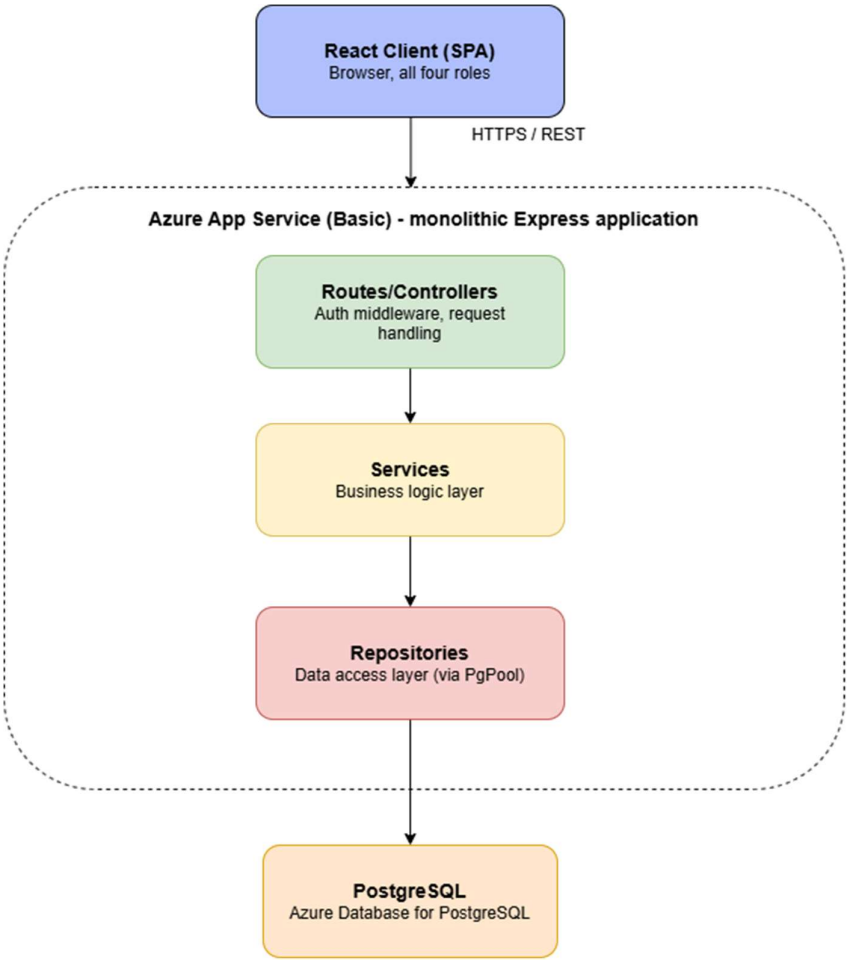


Figure 19: System Architecture Diagram

Cloud Architecture Diagram

The system will be hosted entirely within Microsoft Azure, using the following services:

- Azure App Service (Basic tier): this hosts the Node.js/Express application with the built in React front end served from the same instance as per the Monolithic pattern.
- Azure Database for PostgreSQL (Flexible Server, Burstable tier): this hosts the relational database.
- Custom domain (.org.za) & Let's Encrypt certificate providing HTTPS.
- Network segregation: As there is a project's cost constraints, the Basic App Service tier does not support VNet integration, full private networking is only available from Standard tier upward. Therefore, network segregation is enforced through PostgreSQL server's built-in firewall. Inbound connections are restricted to Azure services and then App services specific outbound IP address, meaning that although the database keeps a public endpoint, only those addresses can connect to it.
- Protocols: All traffic between the client and App service uses HTTPS, TLS via the Let's Encrypt certificate. The connection between the App service and the PostgreSQL server uses SSL-enforced Postgres connections. The Azure Database for PostgreSQL enforces SSL by default. Therefore, data in transit is encrypted on every hop, even without full network isolation. The application does not yet verify the database server's certificate, a known limitation that can be closed by supplying Azure's CA certificate to the connection pool.

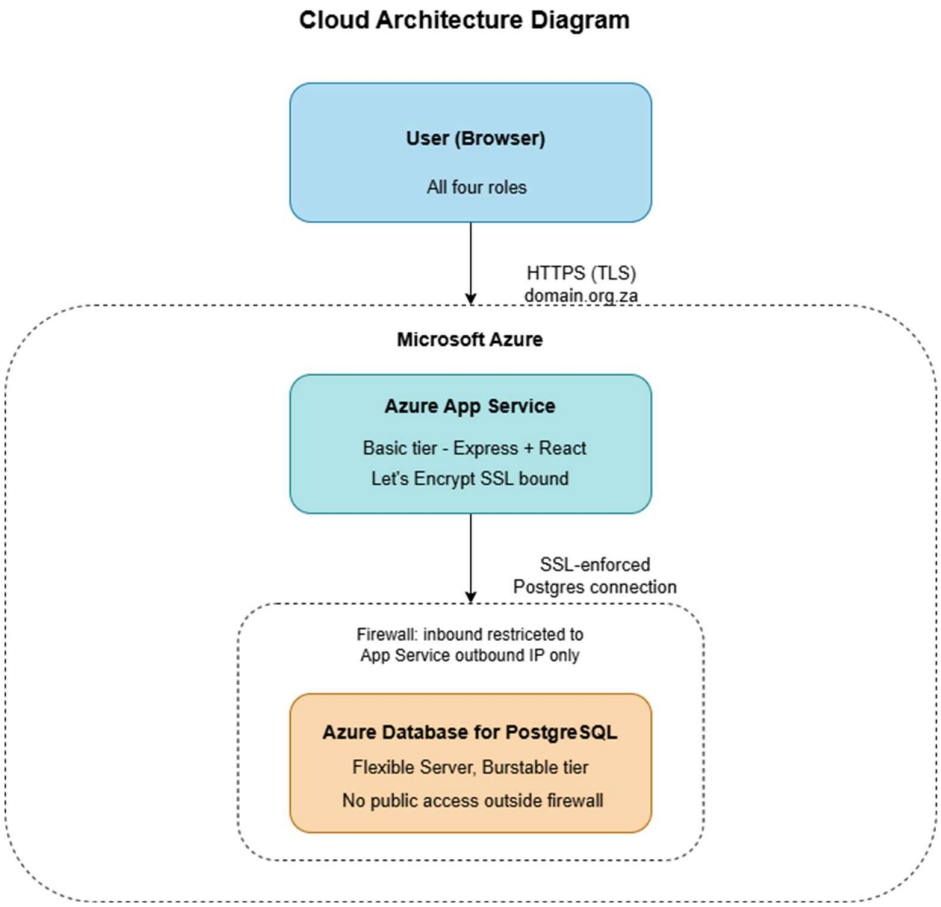


Figure 20: Cloud Architecture Diagram

Security

Security has been viewed as a fundamental requirement rather than an afterthought due to the sensitive nature of the data handled by the Imizamo Yethu Aftercare Management System. This data includes guardian contact information, pickup authorisation records and personal information about minors. This section covers common web vulnerabilities, network security, authentication and authorisation and how the Node.js/Express and Azure-hosted architecture of the system has addressed each.

Network Security

A free Let's Encrypt SSL certificate mapped to the system's custom domain enforces HTTPS encryption for all communication between the client and the Azure App Service instance. Additionally, by default, the connection between the App Service and Azure Database for PostgreSQL is SSL-enforced, which means that data is encrypted not just at the outer edge but at each hop between the browser, server and database. The PostgreSQL server's built-in firewall limits inbound connections to Azure services and the App Service's unique outbound IP address, so although the database has a public endpoint, it only accepts connections from those addresses. This was an explicit, budget-driven trade-off rather than an oversight; full VNet isolation was considered but not chosen because it requires the Standard App Service tier and above, which was not justified against the additional monthly cost for a project at this scale.

Authentication and Authorisation

JSON Web Tokens (JWT) are used for authentication. When the user logs in, the Express backend checks the credentials against the user's bcrypt-hashed and salted password stored in the database and generates a signed JWT with the user's role. This token is stored in an httpOnly, Secure, SameSite=Strict cookie, so it cannot be read by client-side JavaScript (which mitigates theft via XSS) and reduces CSRF exposure without the need for a separate CSRF token scheme.
Authorisation is done via the Express middleware chain (this is the Chain of Responsibility pattern, discussed in the Architecture section). All requests pass through logging, authentication and role-authorisation middleware in sequence before they get to a route handler. For example, a request from a user with the Parent role to a learner-management route passes authentication but is rejected by the role-authorisation middleware with a 403 Forbidden response before it ever reaches the underlying data. This enables applying access control uniformly at a single testable layer instead of being spread out over individual route handlers.

Common Vulnerabilities and Mitigation Strategies

Threat	Description	Mitigation
Cross-Site Scripting (XSS)	An attacker injects malicious scripts into input fields (e.g., a learner observation note or a parent's profile field), which then execute in another user's browser.	The server validates the type, length and format of all input; React escapes rendered content by default, preventing raw HTML/scripts from executing; a Content Security Policy (CSP) header restricts which scripts are permitted to run.
SQL Injection	An attacker manipulates database queries through malicious input.	All queries go through the Repository pattern using node-postgres (pg) parameterised queries (e.g., pool.query('SELECT * FROM learners WHERE id = $1', [id])), so user input is never concatenated directly into SQL.
Broken Access Control	A user accesses data or functions outside their role's permissions.	Enforced centrally through Express role-authorisation middleware, applied consistently to every protected route rather than per-handler.
Cross-Site Request Forgery (CSRF)	An attacker tricks an authenticated user's browser into performing an unintended action.	JWT stored in an httpOnly, Secure, SameSite=Strict cookie, which browsers will not attach to cross-site requests.
Sensitive Data Exposure	Learner and guardian personal information is exposed through weak storage or transmission practices.	TLS is used on every hop (client to App Service to PostgreSQL); the database's public endpoint is restricted by firewall; access is limited by role via middleware.
Credential Attacks (weak/brute-forced passwords)	Attackers attempt to guess or brute-force user passwords.	Passwords are hashed and salted with bcrypt before storage; the login endpoint is rate-limited with express-rate-limit (5 failed attempts per account and 30 per IP address in 15 minutes) to slow brute-force attempts.
Database Connection Exhaustion	A flaw allowing uncontrolled database connections to exhaust PostgreSQL's connection limit, causing denial of service.	Singleton-pattern connection pool (pg.Pool), created once at startup and shared across the whole application, as documented in the Architecture section.

Data Protection and Compliance Considerations

The team has also considered South Africa's Protection of Personal Information Act (POPIA) as the system stores personal information about children. Access to learner and guardian data is strictly limited to what each role needs to perform their function. For example, a teacher can see who may collect a learner, but never the pickup people's phone numbers, and a standard administrator sees only totals, never individual learners. This is in line with good security practice and with POPIA's requirements that the processing of personal information should be limited to what is adequate, relevant and not excessive for a specified purpose.

Summary

Security has been holistically addressed across the stack: transport level security (TLS on every hop, firewall-restricted database), authentication (JWT, bcrypt-hashed passwords), authorisation centralised via Express middleware, XSS, SQL injection and CSRF protection and data security (POPIA) as the system is handling children's personal data.

DevOps

GitHub Actions Pipeline Diagram + Breakdown

This diagram shows how our code is automatically checked, tested, and deployed using GitHub Actions.

---

1. Branching Strategy

We use different branches to keep our work organized:

- **main**- This is the final, stable version of the system. Only fully tested code goes here.
- **Personal branches**- Each team member works on their own branch and opens a pull request into main. There is no develop branch.

This approach stops unfinished or broken code from going into the live system.

---

2. How the Pipeline Works

Every time a team member opens or updates a pull request into main, or a change is merged into main, GitHub Actions automatically starts the following process:

- **Checkout Code** The system downloads the latest version of the code.
- **Install Dependencies** It installs all the required packages and libraries the application needs.
- **Lint the Code** Oxlint checks the React code for common mistakes.
- **Run Automated Tests** The system runs tests to check if the new code works correctly and has not broken existing features. Server tests (Jest and Supertest) run against a temporary PostgreSQL database, and client tests use Vitest and React Testing Library.
- **Build the Application** If the tests pass, the application is built and prepared for deployment.
- **Deploy (only on main branch)** If the code is on the main branch and everything is successful; the system is automatically deployed to the hosting platform.

If any step fails (especially the tests), the pipeline stops and the team is notified so the problem can be fixed.

Why This Is Useful

- It reduces human error.
- It makes sure only working code reaches the final system.
- It saves time because testing and deployment happen automatically.
- It helps the whole team work together more safely.
- This pipeline supports good software development practices and ensures the Aftercare Management System remains stable throughout the project.

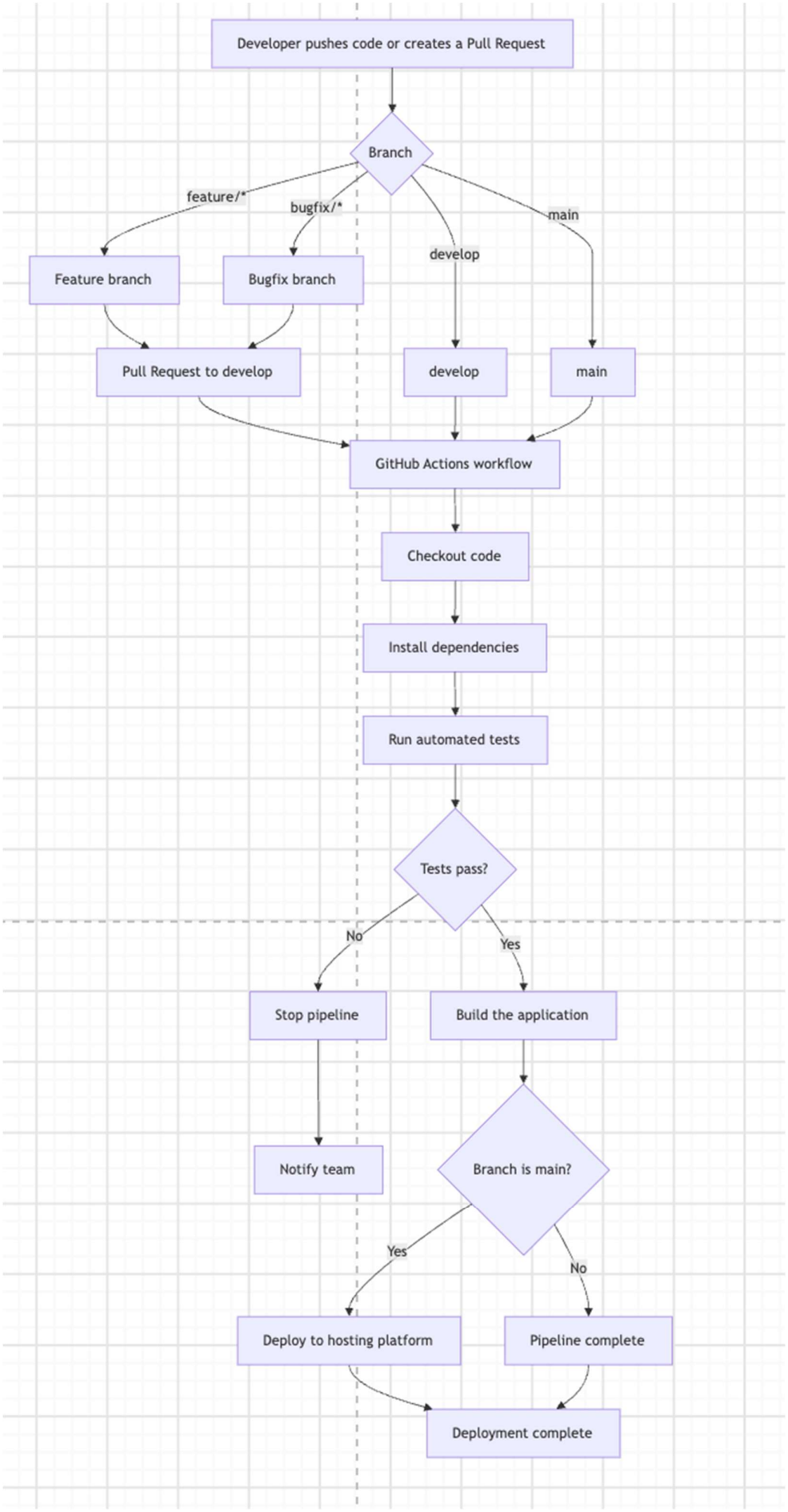


Figure 21: GitHub Actions Pipeline Diagram

Running Costs

Selected Technology Stack

The Imizamo Yethu Aftercare Management System will be deployed using the following technologies:

- Frontend: React
- Backend: Node.js + Express
- Database: PostgreSQL
- Hosting Platform: Azure App Service (Basic Tier)
- SSL Certificate: Let's Encrypt
- Domain Name: .org.za

The application is being developed at no cost by the student development team. Therefore, only the recurring operational costs associated with hosting and maintaining the system are considered in this analysis.

The Imizamo Yethu Aftercare Centre is a non-profit organisation (NPO) and qualifies for annual coverage for its PostgreSQL database. Consequently, the database hosting fee is currently covered at no cost to the organisation. However, the organisation must complete the required registration or renewal process annually to maintain this coverage.

The organisation also already has a registered .org.za domain. Therefore, no initial domain registration cost is included in the current running-cost estimate. Any future renewal fees should be confirmed and budgeted for when applicable.

Estimated Monthly Running Costs

Service	Description	Monthly Cost (ZAR)
Azure App Service (Basic B1)	Web application hosting	R1,000
PostgreSQL Database	Managed database hosting covered by Annual NPO credits	R0
.org.za Domain	Existing registered domain	R0
SSL Certificate (Let's Encrypt)	Secure HTTPS connection	R0
Backups & Monitoring	Basic services included	R0

Estimated Monthly Running Cost: R1 000

Azure Hosting Cost Calculation

Azure App Service Basic (B1) pricing is estimated at approximately USD $54.75 per month. Using an estimated exchange rate of R18.25 per USD, the monthly hosting cost is calculated as follows:

USD $54.75 × R18.25 = approximately R999.19.

This amount is rounded to R1,000 per month for budgeting purposes. Actual costs may vary depending on exchange rates, Azure pricing changes, billing arrangements, and applicable taxes.

Estimated Yearly Running Costs

Service	Annual Cost (ZAR)
Azure App Service (Basic B1)	R12,000
PostgreSQL Database	R0
.org.za Domain Registration	R0
SSL Certificate	R0

Estimated Yearly Running Cost: R12 000

The estimated yearly running cost is R12,000, based on the current hosting estimate.

The PostgreSQL database is covered through the NPO's annual coverage arrangement, provided that the required registration or renewal process is completed each year. The existing domain is also excluded from the current cost estimate because it has already been registered. Any future domain renewal charges should be confirmed separately.

Projected Costs For 120 Users

The selected Azure App Service Basic hosting plan is expected to support this user volume because:

- Attendance data consists primarily of small database records.
- Users are not expected to access the system simultaneously in large numbers.
- The application primarily performs CRUD operations involving attendance, learner records, observations, and reporting.
- Large image, video, and file uploads are not expected to form a significant part of the system's workload.
- PostgreSQL provides relational data storage suitable for the system's structured records.

Based on these requirements, no immediate infrastructure upgrade is anticipated for the current user population.

User Count	Hosting Tier	Monthly Cost (ZAR)	Annual Cost (ZAR)
120 Users	Azure App Service Basic B1	R1 000	R12 000

These figures represent estimated hosting costs under the current deployment arrangement. The PostgreSQL database remains covered at no cost while the organisation maintains its annual NPO coverage.

Future Growth Projection

Although the system is currently designed for approximately 120 users, future growth may require additional hosting resources.

The following table provides an indicative cost projection for larger user populations. These figures are estimates for planning purposes and are not confirmed Azure quotations. Actual costs will depend on application workload, concurrent users, storage requirements, database usage, and the hosting configuration selected.

User Count	Recommended Tier	Monthly Cost (ZAR)	Annual Cost (ZAR)
120 Users	Basic B1	R1,000	R12,000
500 Users	Basic B1 / Standard S1	R1,500	R18,000
1,000 Users	Standard S1	R2,000	R24,000
10,000 Users	Premium Hosting Tier	R4,500+	R54,000+

The projected costs primarily reflect anticipated hosting requirements. The PostgreSQL database is excluded from these estimates under the assumption that the NPO continues to qualify for its annual coverage. If database coverage changes or additional paid resources become necessary, the operational budget must be revised accordingly.

These projections should be reassessed before any hosting upgrade. In particular, the number of registered users alone should not determine the hosting tier. Actual system usage, concurrent access, response times, database performance, and storage requirements should guide infrastructure decisions.

Conclusion

Based on the expected workload of approximately 120 users, the selected technology stack can be operated for an estimated R1,000 per month or R12,000 per year under the current cost assumptions.

The revised estimate reflects the organisation's existing .org.za domain registration and its annual NPO coverage for PostgreSQL. Although the database currently incurs no hosting fee, the organisation must complete the required registration or renewal process each year to maintain its coverage.

The Azure App Service Basic tier provides an affordable starting point for hosting the application, while PostgreSQL offers structured relational data storage for learner records, attendance, guardian information, and reporting.

This arrangement reduces the recurring financial burden on the NPO while providing a practical foundation for the system's current operational requirements. Future infrastructure upgrades should be considered only when actual usage or performance indicates that additional resources are necessary.

Important budgeting consideration: The R12,000 annual estimate represents the currently identified recurring costs. It does not include any future domain renewal charges, unexpected Azure pricing changes, additional paid database resources, or other services that may become necessary. These should be reviewed annually as part of the organisation's operational planning.

Change Management Plan

Overview

The Imizamo Yethu Aftercare Management System is being developed and donated by a student development team to improve the efficiency of daily operations while reducing reliance on paper-based processes. Since the aftercare currently uses manual attendance registers and paper sign-out sheets, the introduction of a digital system represents a significant organisational change.

The primary goal of the change management process is to ensure that staff, management, and other stakeholders are comfortable using the system from the first day of deployment. As technical literacy among teachers is limited and most staff only have access to smartphones, the system has intentionally been designed to be simple, intuitive, and mobile-friendly.

The change management approach will focus not only on initial training, but also on gradual adoption, user support, feedback, and long-term sustainability. This is important because the student development team will eventually hand responsibility for the system to the aftercare. The system therefore needs to be understandable and manageable by staff without requiring continuous involvement from the development team.

Objectives

The adoption strategy aims to:

- Ensure all staff understand why the new system is being introduced.
- Minimise disruption to the daily running of the aftercare.
- Build confidence among teachers who have limited digital experience.
- Replace paper-based attendance and sign-out registers with digital processes.
- Enable management to access reports and student information efficiently.
- Improve the accuracy and accessibility of attendance records.
- Improve child safety by providing a more controlled digital sign-out process.
- Encourage staff to provide feedback about difficulties experienced during adoption.
- Provide appropriate support during the transition from paper-based to digital processes.
- Ensure long-term sustainability after the student development team hands over the system.

Stakeholder Adoption Strategy

Stakeholder	Current Process	Change Introduced	Adoption Strategy
Teachers	Paper attendance and paper sign-out	Smartphone attendance, child sign-out and development tracking	Practical hands-on training using real classroom scenarios.
Aftercare Manager	Paper records and manual reporting	Full system administration and reporting	Advanced training covering administration, reporting and user management.
Parents/Guardians	Paper sign-out and verbal communication	Access to child information and progress (where applicable)	Simple user guide and introduction to the new process.

Board Members	Limited operational involvement	Access to reports and project outcomes	Demonstration of reporting functionality and benefits.
Student Development Team	Developers of the solution	System handover	Provide documentation, training material and technical support during transition.

Different stakeholders will receive different levels of training based on their responsibilities. Teachers require simple, task-focused training because they will primarily use the system for daily attendance and student-related activities. The Aftercare Manager will receive more comprehensive training because they will be responsible for administration, reporting, user management and general oversight of the system.

Adoption Strategy

A phased implementation approach will be used to reduce risk and allow staff to gradually become familiar with the system.

Phase 1 – Awareness

Before deployment, the development team will introduce the system to staff through a demonstration session explaining:

- Why the system has been developed.
- The benefits compared to the previous paper-based process.
- How the system can improve child safety.
- How the system can reduce administrative workload.
- How digital records can improve the accuracy and accessibility of information.
- What changes teachers can expect in their daily routines.
- What support will be available during the transition.

This session should encourage questions and allow staff to express any concerns before implementation. Concerns raised during this stage will be documented and considered before full deployment.

Phase 2 – Training

Teachers and management will participate in guided practical sessions using the actual application.

Training will focus on the daily tasks staff perform, including:

- Logging into the system.
- Recording student attendance.
- Signing students out safely.
- Viewing student safety information.
- Recording child development observations.
- Searching for student profiles.
- Correcting mistakes where permitted.
- Logging out safely.

Rather than teaching technical concepts, the training will concentrate on realistic classroom scenarios to build confidence. Staff will be encouraged to complete tasks themselves rather than simply watching a demonstration.

Training will also allow the development team to identify areas where users experience difficulty. If a particular feature consistently causes confusion, the team can provide additional guidance or consider minor usability improvements.

Phase 3 – Assisted Go-Live

During the first week of implementation:

- Teachers will continue using paper attendance as a backup if required.
- The student development team will be available to answer questions and resolve issues.
- Feedback will be collected from teachers and management.
- Minor usability improvements can be implemented where feasible.
- Staff will be encouraged to report errors or difficulties rather than attempting to work around them.
- The Aftercare Manager will monitor whether attendance and sign-out procedures are being completed correctly.

This reduces the risk associated with moving directly from a fully manual system to a digital one.
The backup paper process will only be temporary. Its purpose is to ensure that attendance and child safety procedures can continue if the system or internet connection becomes temporarily unavailable.

Phase 4 – Full Adoption

Once staff are confident using the application:

- Paper attendance registers will no longer be required for normal operation.
- Digital attendance becomes the official attendance record.
- Digital sign-out replaces paper signature sheets.
- Development tracking becomes part of normal classroom routines.
- Management will use digital reports instead of manually compiling records.
- Staff will be expected to follow the agreed digital procedures consistently.

The Aftercare Manager will be responsible for monitoring continued use and identifying any areas where additional training or support may be required.

Communication Strategy

Effective communication will be maintained throughout the transition to ensure that staff understand what is changing and why.

Stage	Communication	Audience	Purpose
Before deployment	System demonstration	Teachers and Manager	Introduce the system and explain the benefits
Before training	Training schedule and instructions	Teachers and Manager	Prepare users for training
During training	Face-to-face guidance	Teachers and Manager	Build practical skills
Go-live	Daily check-ins	Teachers and Manager	Identify issues early
After go-live	Feedback session	Teachers and Manager	Evaluate adoption
Handover	Formal walkthrough	Manager	Transfer responsibility
Post-handover	Documentation and support information	Manager	Support long-term sustainability

Communication should use straightforward language and avoid unnecessary technical terminology. Since the users have varying levels of technical experience, demonstrations and visual instructions should be prioritised over lengthy technical explanations.

Feedback and Evaluation

User feedback will be collected throughout implementation to determine whether the system is achieving its intended objectives.

Feedback can be collected through:

- Informal discussions with teachers.
- Short feedback forms.
- Discussions with the Aftercare Manager.
- Observation of common difficulties during the first week.
- Recording frequently asked questions.
- Reviewing system usage and common errors.

The feedback will be used to identify:

- Features that users find difficult to understand.
- Training topics that require additional explanation.
- Technical problems.
- Areas where the user interface could be improved.
- Whether the system is reducing reliance on paper.
- Whether attendance and sign-out processes are being completed consistently.

The development team will prioritise issues that affect child safety, attendance accuracy, data integrity, and the ability of staff to perform their daily responsibilities**.**

Risk Management During Adoption

Risk	Potential Impact	Mitigation
Staff struggle to use the system	Low adoption and incorrect records	Practical training and simple user guides
Internet connection is unavailable	Attendance cannot be recorded digitally	Temporary paper backup procedure
Staff forget new procedures	Inconsistent use of the system	Reminders and follow-up training
Incorrect information is entered	Inaccurate student records	Training, validation and manager oversight
Staff are resistant to change	Continued reliance on paper	Demonstrate benefits and involve staff in feedback
Development team is no longer available	Difficulties after handover	Detailed documentation and manager training
Device or phone problems	Individual teacher unable to access system	Shared backup device or temporary paper procedure
Passwords are forgotten	Users cannot access the system	The Master Administrator resets forgotten staff passwords; parents cannot yet reset their own, so the centre asks the technical maintainer

Handover Plan

As the software is being donated, ownership will be transferred to Imizamo Yethu Aftercare upon project completion.

The following deliverables will be provided during handover:

- Fully deployed web application.
- Manager account for the Aftercare Manager, and the Master Administrator account for the client's nominated representative.
- Source code repository.
- Technical documentation.
- User manual.
- Administrator manual.
- Database documentation.
- Installation and deployment guide.
- Backup and recovery instructions.
- Basic troubleshooting guide.
- System credentials and configuration information where appropriate.

The development team will also provide a walkthrough of the system with the Aftercare Manager to ensure they are comfortable managing users, maintaining student information and generating reports. The handover will only be considered complete once the Aftercare Manager has demonstrated that they can perform the key administrative tasks without direct assistance from the development team.

Training Plan

Teacher Training

Audience: Class teachers and substitute teachers
Duration: Approximately 2 hours

Topics

- Logging into the system.
- Recording attendance using a smartphone.
- Searching for students.
- Viewing important child information.
- Recording development observations.
- Completing safe digital student sign-out.
- Correcting permitted mistakes.
- Logging out securely.
- What to do if the system is unavailable.

Training Method

- Live demonstration.
- Guided practical exercises.
- Scenario-based activities.
- Question and answer session.
- Short practical competency check.

At the end of the session, teachers should be able to independently complete the normal attendance and sign-out process.

Manager Training

Audience: Aftercare Manager
Duration: Approximately 3 hours

Topics

- System administration.
- Adding, suspending and restoring teacher accounts.
- Editing student records.
- Managing classes and assigning teachers.
- Viewing attendance statistics.
- Generating donor reports.
- Exporting PDF reports.
- Approving or dismissing flagged collections in the sign-out queue.
- Basic troubleshooting.
- Password and account management.
- Backup procedures.
- Understanding the handover documentation.

The Manager will receive additional practical exercises because they will become the primary internal administrator after the development team's involvement ends.

Parent Orientation

Parents will not require formal classroom training.

Instead, they will receive:

- A short information sheet explaining the new system.
- Instructions for accessing their child's information.
- An explanation of the new digital sign-out process.
- Basic guidance on whom to contact if they experience difficulties.

The purpose of parent communication is to ensure that parents understand the reason for the change and are aware of how the new system affects them.

Post-Handover Support

The student development team will provide a short transition period following deployment to assist the Aftercare with problems that may arise during early adoption.

During this period, support will focus on:

- Clarifying system functionality.
- Assisting with user access issues.
- Identifying technical errors.
- Answering questions about the documentation.
- Recording recurring issues for possible resolution.

After the formal support period ends, the Aftercare Manager will become the primary point of contact for internal user support. The system documentation will allow the Aftercare to continue operating the system without depending on the original student developers.

Sustainability Plan

Long-term sustainability is an important part of the change management process because the system is being donated to the NPO.

To support continued use:

- The Aftercare Manager will be responsible for system administration.
- At least one additional staff member should be familiar with basic administrative procedures where possible.
- User and administrator manuals will remain available to staff.
- Backup procedures will be documented and followed regularly.
- New staff members should receive a basic system orientation before using the system independently.
- The Aftercare should maintain a record of technical issues and recurring user problems.
- Any future development should be documented so that changes remain understandable to future administrators.

This approach reduces the risk that the system becomes unusable when staff members change or when the original development team is no longer available.

Change Success Measures

The success of the change management process will be evaluated using several practical measures:

Success Measure	**Target**
Staff training completion	All relevant staff trained before full adoption
Attendance adoption	Digital attendance used as the primary process
Sign-out adoption	Digital sign-out replaces paper sign-out
Staff confidence	Staff can complete normal tasks independently
Manager competency	Manager can perform core administrative functions
Paper dependency	Significant reduction or elimination of routine paper records
User feedback	Major usability issues identified and addressed
System sustainability	Aftercare can operate the system without continuous developer assistance

The change will be considered successful when staff can independently use the system as part of their normal daily activities, management can administer the system effectively, and the aftercare no longer relies on the student development team for routine operation.

Feedback Implementations

Lecturer feedback was heavily considered throughout the development life-cycle, following our Part 1 and Part 2 submissions.

Declarations of Authenticity

POE Part 1 + 2

Declaration of authenticity

I, Michaela Ferraris ID Number, 0409260118081 hereby declare that this portfolio, and any evidence included therein, contains my own independent work and that I have not received help from other groups.

I confirm that we have not committed plagiarism in the accomplishment of this work, nor have I falsified and/or invented experimental data.

I accept the academic penalties that may be imposed for violations of the above.

STUDENT NUMBER: ST10325652
STUDENT NAME: Michaela Ferraris
STUDENT SIGNATURE: _____ DATE: 17/08/2026

Declaration of Authenticity

I, Thomas Pheiffer ID Number, 0303235071089 hereby declare that this portfolio, and any evidence included therein, contains my own independent work and that I have not received help from other groups.

I confirm that we have not committed plagiarism in the accomplishment of this work, nor have I falsified and/or invented experimental data.

I accept the academic penalties that may be imposed for violations of the above.

STUDENT NUMBER: ST10068995

STUDENT NAME: Thomas Pheiffer

STUDENT SIGNATURE: Thomas Pheiffer DATE: 17 August 2026

I, Justin Fussell ID Number, BE988112 hereby declare that this portfolio, and any evidence included therein, contains my own independent work and that I have not received help from other groups.

I confirm that we have not committed plagiarism in the accomplishment of this work, nor have I falsified and/or invented experimental data.

I accept the academic penalties that may be imposed for violations of the above.

STUDENT NUMBER: ST10280758
STUDENT NAME: Justin Fussell
STUDENT SIGNATURE: JF DATE: 17/08/2026

Declaration of authenticity

I, Isabelle Kathryn Devlin ID Number, 0511060679087 hereby declare that this portfolio, and any evidence included therein, contains my own independent work and that I have not received help from other groups.

I confirm that we have not committed plagiarism in the accomplishment of this work, nor have I falsified and/or invented experimental data.

I accept the academic penalties that may be imposed for violations of the above.

STUDENT NUMBER: ST10445500
STUDENT NAME: Isabelle Kathryn Devlin
STUDENT SIGNATURE: DATE: 17 August 2026

Declaration of authenticity

I, Nomsa Mkoka ID Number, 0410110527083 hereby declare that this portfolio, and any evidence included therein, contains my own independent work and that I have not received help from other groups.

I confirm that we have not committed plagiarism in the accomplishment of this work, nor have I falsified and/or invented experimental data.

I accept the academic penalties that may be imposed for violations of the above.

STUDENT NUMBER: ST10301146
STUDENT NAME: Nomsa Mkoka
STUDENT SIGNATURE: _____ **DATE:** 2026/08/17

Prototype

Figma Link

Link: < https://www.figma.com/design/tbqMsP2EFvGegvw3yWtPNo/Aftercare-Management-System?node-id=103-8>

If the link above does not work, a PDF of the exported Figma project will be submitted alongside this document.

Proof of Collaboration

Multiple meetings were held with all team members present. Either held online or in-person on campus, team meetings were arranged for the purpose of task management and allocation, sharing updates, and progress tracking. A WhatsApp group chat was also made for quick communication between members outside of meetings. As for communication with the client, online meetings were held with all members. One team member was allocated to taking down meeting minutes, while the rest focused on presenting.

As for document sharing and management, a shared Google Drive stored all work made by team members as well as module and client information.

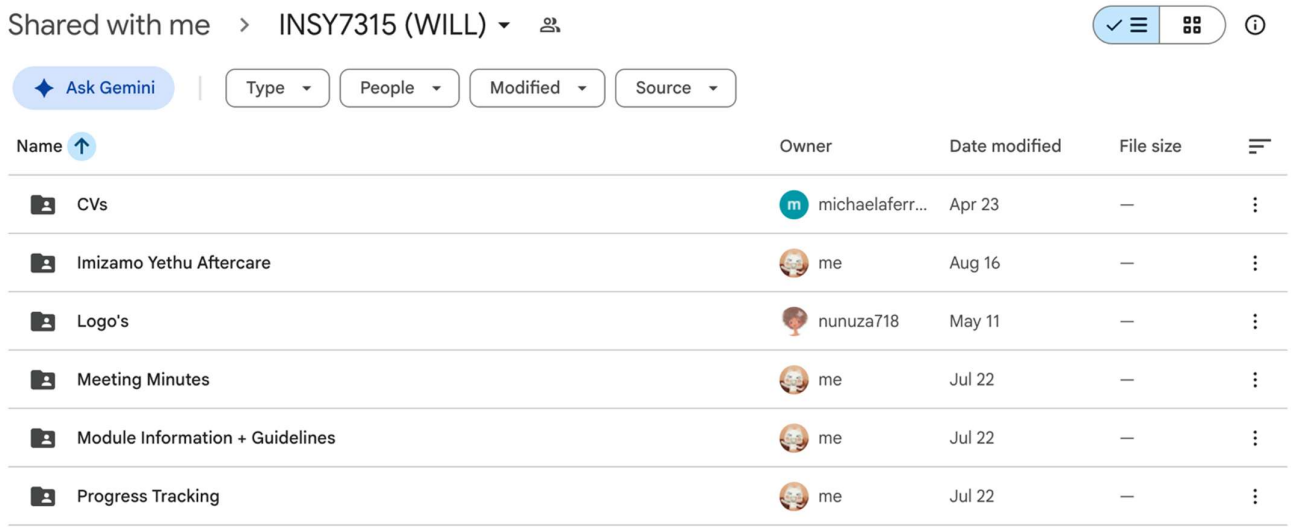


Figure 22: Team Shared Google Drive

Further documentation was made to allow for seamless task allocation, management, and progress tracking. A detailed spreadsheet has been made for each part of the overall project, as seen below:

Imizamo Yethu Aftercare Progress Tracking

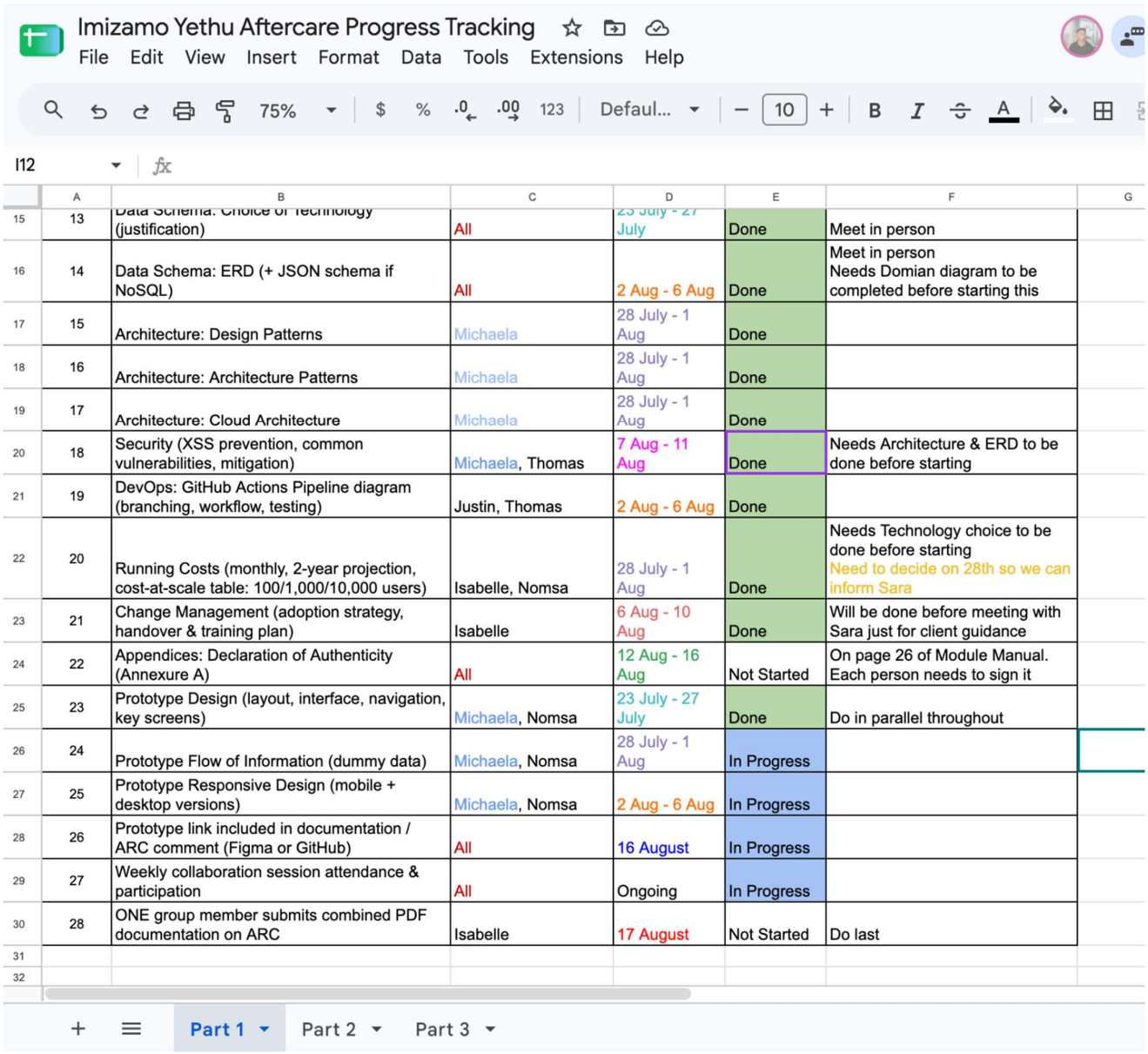
File Edit View Insert Format Data Tools Extensions Help

I12

Task 1 - Documentation 80% - 17 August 2026					
Number	Requirement/Deliverable	Person Responsible	Target Date	Status	Comments
1	Cover Page: Team Number, Team Name, member names & student numbers, client name & logo	Isabelle	23 July - 27 July	Done	Will be attached to shared drive as word doc at later date for submission.
2	Table of Contents (numbered sections, figures/tables)	Isabelle	12 August - 16 August	Done	Do last
3	Introduction (justification of system for management)	Isabelle	12 August - 16 August	Done	Do last
4	Requirements: User Roles	Justin, Thomas	23 July - 27 July	Done	
5	Requirements: User Stories (standard format)	Justin, Thomas	28 July - 1 Aug	Done	Needs user roles to be done before starting
6	Requirements: UX Journey Map (screens & navigation per role)	Justin, Thomas	2 Aug - 6 Aug	Done	Meet in person Needs user roles and stories to be done before starting
7	Requirements: Work Breakdown Structure (tasks, description, duration, predecessors, team members, resources)	Nomsa, Thomas	23 July - 1 Aug	Done	
8	Non-Functional Requirements	Michaela	23 July - 27 July	Done	
9	Analysis Artifacts: Domain Class Diagram	Isabelle, Nomsa	28 July - 1 Aug	Done	
10	Analysis Artifacts: Design Class Diagram	Isabelle, Nomsa	2 Aug - 6 Aug	Done	Needs Domian diagram to be completed before starting this
11	Implementation: Sequence & State Diagrams (critical flows)	Justin	7 Aug - 11 Aug	Done	Needs Design class diagram to be completed before starting this
12	Implementation: Deployment Plan	Michaela	7 Aug - 11 Aug	Done	Needs technology choice done before starting this Does anyone know where I can find what needs to go in this plan?
13	Data Schema: Choice of Technology (justification)	All	23 July - 27 July	Done	Meet in person
					Meet in person

Part 1 Part 2 Part 3

Figure 23: Part One of Shared Progress Tracking Spreadsheet



15	13	Data Schema: Choice of Technology (justification)	All	23 July - 27 July	Done	Meet in person
16	14	Data Schema: ERD (+ JSON schema if NoSQL)	All	2 Aug - 6 Aug	Done	Meet in person Needs Domian diagram to be completed before starting this
17	15	Architecture: Design Patterns	Michaela	28 July - 1 Aug	Done	
18	16	Architecture: Architecture Patterns	Michaela	28 July - 1 Aug	Done	
19	17	Architecture: Cloud Architecture	Michaela	28 July - 1 Aug	Done	
20	18	Security (XSS prevention, common vulnerabilities, mitigation)	Michaela, Thomas	7 Aug - 11 Aug	Done	Needs Architecture & ERD to be done before starting
21	19	DevOps: GitHub Actions Pipeline diagram (branching, workflow, testing)	Justin, Thomas	2 Aug - 6 Aug	Done	
22	20	Running Costs (monthly, 2-year projection, cost-at-scale table: 100/1,000/10,000 users)	Isabelle, Nomsa	28 July - 1 Aug	Done	Needs Technology choice to be done before starting Need to decide on 28th so we can inform Sara
23	21	Change Management (adoption strategy, handover & training plan)	Isabelle	6 Aug - 10 Aug	Done	Will be done before meeting with Sara just for client guidance
24	22	Appendices: Declaration of Authenticity (Annexure A)	All	12 Aug - 16 Aug	Not Started	On page 26 of Module Manual. Each person needs to sign it
25	23	Prototype Design (layout, interface, navigation, key screens)	Michaela, Nomsa	23 July - 27 July	Done	Do in parallel throughout
26	24	Prototype Flow of Information (dummy data)	Michaela, Nomsa	28 July - 1 Aug	In Progress	
27	25	Prototype Responsive Design (mobile + desktop versions)	Michaela, Nomsa	2 Aug - 6 Aug	In Progress	
28	26	Prototype link included in documentation / ARC comment (Figma or GitHub)	All	16 August	In Progress	
29	27	Weekly collaboration session attendance & participation	All	Ongoing	In Progress	
30	28	ONE group member submits combined PDF documentation on ARC	Isabelle	17 August	Not Started	Do last

Figure 24: Part 2 of Progress Tracking Spreadsheet

Meetings were held on the following dates, each averaging a period of 40 minutes to 1h30 minutes.

- **23 April 2026** – First Online Team Meeting
- **30 April 2026** – Online Team Meeting
- **10 May 2026** – In-person Team Meeting
- **04 June 2026** – Online Team Meeting
- **24 June 2026** – Online Meeting with Sara, Board Chairman for Imizamo Yethu
- **26 June 2026** – Online Team Meeting
- **30 June 2026** – Online Meeting with other potential client
- **22 July 2026** – Online Team Meeting
- **23 July 2026** – In-person Team Meeting
- **3 August 2026** – In-Person Team Meeting
- **4 August 2026** – Online Meeting with Sara, Board Chairman for Imizamo Yethu
- **6 August 2026** – In-person Team Meeting
- **11 August 2026** – In-Person/Online Team Meeting
- **17 August 2026** – Final In-Person Meeting for submission

---

References

Microsoft (2024). *Azure documentation*. [online] *learn.microsoft.com*. Available at: https://learn.microsoft.com/en-us/azure/?product=popular [Accessed 2026].

Visual Studio Code (2023). *Documentation for Visual Studio Code*. [online] *code.visualstudio.com*. Available at: https://code.visualstudio.com/docs [Accessed 2026].

The PostgreSQL Global Development Group (2025). *PostgreSQL: Documentation*. [online] *www.postgresql.org*. Available at: https://www.postgresql.org/docs/ [Accessed 2026].

AI Declaration

Artificial Intelligence (AI) tools were used in a limited capacity during this project to assist with general research, idea generation, and the refinement of written content. AI served only as a supporting tool and did not replace the work of the project team. All key decisions, analysis, design, development, and final deliverables were completed, reviewed, and validated by the team members. The project's outcomes, conclusions, and submitted work remain the original work of the team.

Link: https://claude.ai/share/43516dc0-c5be-4942-a566-f3861f5c1228
